FILE: requirements.txt (Core Dependencies)

```
# OmniSign 3D - Two-Way American Sign Language (ASL) Translator Dependencies

Flask>=3.0.0

flask-cors>=4.0.0

numpy>=1.24.0,<2.0.0

pandas>=2.0.0

tensorflow>=2.16.0

tf_keras>=2.16.0

opencv-python>=4.8.0

mediapipe>=0.10.14

requests>=2.31.0

scipy>=1.11.0

h5py>=3.10.0

python-pptx>=1.0.0

python-docx>=1.1.0

reportlab>=4.0.0

Pygments>=2.15.0
```

FILE: app.py (Backend REST API, Neural Ensemble & Translation Logic)

```
import os

os.environ["TF_USE_LEGACY_KERAS"] = "1"

os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"

import time

import json

import tempfile

import cv2

import mediapipe as mp


import numpy as np

import pandas as pd

from flask import Flask, request, jsonify, send_from_directory

from flask_cors import CORS

from tensorflow.keras.models import load_model


app = Flask(__name__, static_folder='.', template_folder='.')

CORS(app, resources={r"/api/*": {"origins": "*"}})


BASE_DIR = os.path.dirname(os.path.abspath(__file__))

DATASET_DIR = os.path.join(BASE_DIR, "dataset", "split_features", "train")

STATIC_DATASET_DIR = os.path.join(BASE_DIR, "static_dataset")

CLASSES_JSON_PATH = os.path.join(BASE_DIR, "classes.json")

CLASSES_2000_PATH = os.path.join(BASE_DIR, "classes_2000.json")


# 1. Load Vocab / Classes (Classes.json primary for portable Git execution)

classes = []

if os.path.exists(CLASSES_JSON_PATH):

    try:

        with open(CLASSES_JSON_PATH, "r", encoding="utf-8") as f:

            classes = json.load(f)

    except Exception as e:

        print(f"Notice: Failed to read classes.json: {e}")


classes_2000 = []

if os.path.exists(CLASSES_2000_PATH):

    try:

        with open(CLASSES_2000_PATH, "r", encoding="utf-8") as f:

            classes_2000 = json.load(f)

    except Exception as e:

        print(f"Notice: Failed to read classes_2000.json: {e}")


if not classes and os.path.exists(DATASET_DIR):

    classes = sorted([d for d in os.listdir(DATASET_DIR) if os.path.isdir(os.path.
      join(DATASET_DIR, d))])

elif not classes and os.path.exists(STATIC_DATASET_DIR):

    classes = sorted([f.replace(".json", "") for f in os.listdir(
      STATIC_DATASET_DIR) if f.endswith(".json") and f != "hello.json"])


label_map = {i: word for i, word in enumerate(classes)}


# Vocabulary Categories for UI Filtering

CATEGORIES = {

    "Greetings & Basics": ["hello", "fine", "yes", "no", "help", "now", "later", "
      right", "wrong", "same"],

    "Questions & Communication": ["what", "who", "how", "tell", "language", "
      letter", "meet", "hearing", "deaf"],

    "Actions & Verbs": ["eat", "drink", "walk", "dance", "play", "study", "work",
      "give", "go", "pull", "cook", "paint", "change", "decide", "finish", "forget
      ", "want", "need", "like", "enjoy"],

    "People & Family": ["family", "mother", "son", "woman", "man", "doctor", "
      secretary", "cousin"],

    "Objects & Things": ["book", "computer", "chair", "table", "bed", "clothes", "
      shirt", "jacket", "hat", "paper", "medicine"],

    "Food & Dining": ["apple", "candy", "pizza", "fish", "corn", "drink", "eat"],

    "Time & Colors": ["time", "year", "thursday", "birthday", "thanksgiving", "
      black", "blue", "brown", "color", "orange", "pink", "purple", "white", "dark
      ", "cool", "hot"],

    "Animals & Nature": ["bird", "cow", "dog", "city", "africa"],

    "Descriptions": ["full", "tall", "short", "thin", "last", "before", "many", "
      all"]

}


# 2. Load Models (Ensemble for maximum accuracy)

model_transformer = None

model_bilstm = None


TRANS_2000_MODEL_PATH = os.path.join(BASE_DIR, "transformer_wlasl_2000.h5")

TRANS_MODEL_PATH = os.path.join(BASE_DIR, "transformer_wlasl_100.h5")

LSTM_MODEL_PATH = os.path.join(BASE_DIR, "bilstm_wlasl_100_optimized.h5")


if os.path.exists(TRANS_2000_MODEL_PATH):

    try:

        print(f"Loading WLASL-2000 Transformer model from {TRANS_2000_MODEL_PATH}
          ...")

        model_transformer = load_model(TRANS_2000_MODEL_PATH)

        if classes_2000:

            classes = classes_2000

            label_map = {i: word for i, word in enumerate(classes)}

        print(f"WLASL-2000 Transformer loaded successfully ({len(classes)}
          classes).")

    except Exception as e:

        print(f"Failed to load WLASL-2000 Transformer: {e}")


if model_transformer is None and os.path.exists(TRANS_MODEL_PATH):

    try:

        print(f"Loading Benchmark Transformer model from {TRANS_MODEL_PATH}...")

        model_transformer = load_model(TRANS_MODEL_PATH)

        print("Transformer loaded successfully.")

    except Exception as e:

        print(f"Failed to load Transformer: {e}")
```

```
if os.path.exists(LSTM_MODEL_PATH):
    try:
        print(f"Loading BiLSTM model from {LSTM_MODEL_PATH}...")
        model_bilstm = load_model(LSTM_MODEL_PATH)
        print("BiLSTM loaded successfully.")
    except Exception as e:
        print(f"Failed to load BiLSTM: {e}")


def normalize_sequence(sequence):
    """
    Centers coordinates to the nose and scales by shoulder width.
    Matches the exact training normalization pipeline.
    """
    normalized_seq = []
    for frame in sequence:
        if np.all(frame == 0):
            normalized_seq.append(frame)
            continue

        nose_x, nose_y, nose_z = frame[0], frame[1], frame[2]
        ls_x, ls_y = frame[33], frame[34]
        rs_x, rs_y = frame[36], frame[37]

        shoulder_width = np.sqrt((ls_x - rs_x)**2 + (ls_y - rs_y)**2)
        if shoulder_width == 0:
            shoulder_width = 1.0

        new_frame = np.zeros_like(frame)
        for i in range(0, len(frame), 3):
            if not (frame[i] == 0 and frame[i+1] == 0 and frame[i+2] == 0):
                new_frame[i] = (frame[i] - nose_x) / shoulder_width
                new_frame[i+1] = (frame[i+1] - nose_y) / shoulder_width
                new_frame[i+2] = (frame[i+2] - nose_z) / shoulder_width

        normalized_seq.append(new_frame)
    return np.array(normalized_seq)


def interpolate_sequence(sequence_array):
    """Interpolates dropped or missing landmarks smoothly across frames."""
    df = pd.DataFrame(sequence_array).replace(0.0, np.nan)
    return df.interpolate(method='linear', limit_direction='both').fillna(0.0).
      to_numpy()


@app.route('/')
def home():
    return send_from_directory('.', 'index.html')


@app.route('/<path:filename>')
def serve_static(filename):
    return send_from_directory('.', filename)


@app.route('/api/status', methods=['GET'])
def get_status():
    total_3d = len([f for f in os.listdir(STATIC_DATASET_DIR) if f.endswith('.json
      ')]) if os.path.exists(STATIC_DATASET_DIR) else len(classes)
    is_2000 = (model_transformer is not None and getattr(model_transformer, '
      output_shape', [None, 0])[-1] == 2000)
    return jsonify({
        "status": "online",
        "models": {
            "transformer": model_transformer is not None,
            "bilstm": model_bilstm is not None,
            "ensemble_active": (model_transformer is not None and model_bilstm is
              not None),
            "wlasl_2000_active": is_2000
        },
        "ensemble_weights": {"transformer": 0.65, "bilstm": 0.35},
        "num_classes": len(classes),
        "total_dataset_classes": len(classes_2000) if classes_2000 else 2000,
        "total_3d_signs": total_3d,
        "input_shape": [60, 225]
    })


@app.route('/api/health', methods=['GET'])
def health_check():
    return jsonify({
        "status": "healthy",
        "timestamp": time.time(),
        "models_loaded": {
            "transformer": model_transformer is not None,
            "bilstm": model_bilstm is not None
        }
    })


@app.route('/api/vocabulary', methods=['GET'])
@app.route('/api/classes', methods=['GET'])
def get_vocabulary():
    signs_3d = []
    if os.path.exists(STATIC_DATASET_DIR):
        signs_3d = sorted([f.replace(".json", "") for f in os.listdir(
          STATIC_DATASET_DIR) if f.endswith(".json")])

    all_vocab = classes_2000 if classes_2000 else classes
    return jsonify({
        "count": len(classes),
        "total_dataset_classes": len(all_vocab),
        "total_3d_signs": len(signs_3d),
        "classes": classes,
        "all_classes": sorted(all_vocab),
        "signs_3d": signs_3d,
        "categories": CATEGORIES
    })


# Endpoint 1: ASL -> Text (Inference with Ensemble & Top-K)
@app.route('/api/predict', methods=['POST'])
def predict():
    t_start = time.time()
    payload = request.get_json(silent=True) or {}
    data = payload.get("landmarks")

    if not data or len(data) == 0:
        return jsonify({"error": "No landmark sequence provided"}), 400
```

```
    try:
        seq = np.array(data, dtype=np.float32)
    except Exception as e:
        return jsonify({"error": f"Failed to parse landmarks array: {str(e)}"}),
          400

    if seq.ndim != 2 or seq.shape[1] != 225:
        return jsonify({"error": f"Invalid landmark shape {seq.shape}
          . Expected (N, 225)"}), 400

    # Handle length adjustments to 60 frames
    if len(seq) > 60:
        seq = seq[-60:]
    elif len(seq) < 60:
        padding = np.zeros((60 - len(seq), 225), dtype=np.float32)
        seq = np.vstack((seq, padding))

    # 1. Temporal Interpolation
    smoothed = interpolate_sequence(seq)

    # 2. Spatial Normalization (Centers to nose, scales to shoulder width)
    normalized = normalize_sequence(smoothed)
    input_tensor = np.expand_dims(normalized, axis=0)

    # 3. Model Inference (Ensemble for maximum accuracy)
    preds = None
    if model_transformer is not None and model_bilstm is not None:
        p_trans = model_transformer.predict(input_tensor, verbose=0)[0]
        p_lstm = model_bilstm.predict(input_tensor, verbose=0)[0]
        # Weighted Ensemble: 65% Transformer + 35% BiLSTM
        preds = 0.65 * p_trans + 0.35 * p_lstm
    elif model_transformer is not None:
        preds = model_transformer.predict(input_tensor, verbose=0)[0]
    elif model_bilstm is not None:
        preds = model_bilstm.predict(input_tensor, verbose=0)[0]
    else:
        return jsonify({"error": "No trained model currently available on server"}
          ), 503

    # 4. Top-K Ranking
    top_indices = np.argsort(preds)[::-1][:5]
    top_predictions = [
        {
            "word": label_map.get(int(idx), f"Class {idx}"),
            "confidence": float(preds[idx]),
            "percent": round(float(preds[idx]) * 100, 1)
        }
        for idx in top_indices
    ]

    best_idx = int(top_indices[0])
    best_confidence = float(preds[best_idx])
    latency_ms = round((time.time() - t_start) * 1000, 1)

    return jsonify({
        "prediction": label_map.get(best_idx, "Unknown"),
        "confidence": best_confidence,
        "percent": round(best_confidence * 100, 1),
        "top_predictions": top_predictions,
        "latency_ms": latency_ms
    })


# ================ SENTENCE NLP & KINEMATIC COMPILER ================
SYNONYMS = {
    # Greetings & Politeness
    "hi": "hello", "hey": "hello", "greetings": "hello", "welcome": "hello",
    "okay": "fine", "ok": "fine", "good": "fine", "great": "fine", "alright": "
      fine",
    "yeah": "yes", "yep": "yes", "sure": "yes",
    "nope": "no", "nah": "no", "negative": "no",
    "assist": "help", "assistance": "help", "aid": "help",
    # Verbs
    "eats": "eat", "eating": "eat", "ate": "eat", "eaten": "eat", "food": "eat", "
      meal": "eat", "dinner": "eat", "lunch": "eat", "breakfast": "eat",
    "drinks": "drink", "drinking": "drink", "drank": "drink", "drunk": "drink", "
      water": "drink", "beverage": "drink",
    "walks": "walk", "walking": "walk", "walked": "walk", "stroll": "walk",
    "dances": "dance", "dancing": "dance", "danced": "dance",
    "plays": "play", "playing": "play", "played": "play", "game": "play",
    "studies": "study", "studying": "study", "studied": "study", "learn": "study",
       "learning": "study",
    "works": "work", "working": "work", "worked": "work", "job": "work",
    "gives": "give", "giving": "give", "gave": "give", "given": "give",
    "goes": "go", "going": "go", "went": "go", "gone": "go", "leave": "go",
    "pulls": "pull", "pulling": "pull", "pulled": "pull",
    "cooks": "cook", "cooking": "cook", "cooked": "cook",
    "paints": "paint", "painting": "paint", "painted": "paint",
    "changes": "change", "changing": "change", "changed": "change",
    "decides": "decide", "deciding": "decide", "decided": "decide",
    "finishes": "finish", "finishing": "finish", "finished": "finish", "done": "
      finish",
    "forgets": "forget", "forgetting": "forget", "forgot": "forget", "forgotten":
      "forget",
    "wants": "want", "wanting": "want", "wanted": "want", "wish": "want",
    "needs": "need", "needing": "need", "needed": "need", "require": "need",
    "likes": "like", "liking": "like", "liked": "like", "love": "like",
    "enjoys": "enjoy", "enjoying": "enjoy", "enjoyed": "enjoy",
    "tells": "tell", "telling": "tell", "told": "tell", "say": "tell", "speak": "
      tell",
    "meets": "meet", "meeting": "meet", "met": "meet",
    # People & Family
    "families": "family", "household": "family",
    "mom": "mother", "mommy": "mother", "mama": "mother", "female": "woman",
    "dad": "man", "daddy": "man", "father": "man", "boy": "man", "guy": "man",
    "women": "woman", "girl": "woman", "lady": "woman",
    "men": "man", "gentleman": "man",
    "sons": "son", "kid": "son", "child": "son", "children": "son",
    "doctors": "doctor", "physician": "doctor", "medic": "doctor",
    "secretaries": "secretary", "assistant": "secretary",
    "cousins": "cousin", "relative": "cousin",
    # Objects
    "books": "book", "reading": "book",
    "computers": "computer", "laptop": "computer", "pc": "computer",
    "chairs": "chair", "seat": "chair",
    "tables": "table", "desk": "table",
```

```
    "beds": "bed", "mattress": "bed", "sleep": "bed",
    "clothes": "clothes", "clothing": "clothes", "garment": "clothes",
    "shirts": "shirt", "top": "shirt",
    "jackets": "jacket", "coat": "jacket",
    "hats": "hat", "cap": "hat",
    "papers": "paper", "document": "paper",
    "medicines": "medicine", "pill": "medicine", "drugs": "medicine",
    # Food
    "apples": "apple", "candies": "candy", "sweet": "candy", "sweets": "candy",
    "pizzas": "pizza", "fishes": "fish", "corns": "corn",
    # Animals & Places
    "birds": "bird", "cows": "cow", "cattle": "cow", "dogs": "dog", "puppy": "dog"
      , "puppies": "dog",
    "cities": "city", "town": "city", "village": "city",
    # Time & Modifiers
    "times": "time", "clock": "time", "hour": "time",
    "years": "year", "annual": "year",
    "colors": "color",
    "correct": "right", "true": "right",
    "incorrect": "wrong", "false": "wrong",
    "lots": "many", "much": "many",
    "every": "all", "everything": "all", "everyone": "all"
}

STOP_WORDS = {
    'i', 'me', 'my', 'you', 'your', 'he', 'she', 'it', 'we', 'they', 'them',
    'is', 'am', 'are', 'was', 'were', 'be', 'been', 'being',
    'the', 'a', 'an', 'to', 'of', 'in', 'on', 'at', 'by', 'for', 'with', 'about',
    'into', 'through', 'during', 'above', 'below', 'from', 'up', 'down', 'out',
    'off', 'over', 'under', 'then', 'so', 'than', 'too', 'very', 'just', 'do', '
      does', 'did',
    'please', 'and'
}

def blend_motion_frames(f1, f2, steps=8):
      """Generates smooth ease-in-out transitional frames between consecutive signs."""
    blended = []
    a1 = np.array(f1, dtype=np.float32)
    a2 = np.array(f2, dtype=np.float32)
    for s in range(1, steps + 1):
        t = s / (steps + 1.0)
        alpha = 3.0 * t * t - 2.0 * t * t * t
        frame = (1.0 - alpha) * a1 + alpha * a2
        blended.append(frame.tolist())
    return blended

def parse_text_to_asl_glosses(raw_text):
    """Transforms raw English sentence into ASL gloss sequence."""
    cleaned = ''.join(c.lower() if c.isalnum() or c.isspace() else ' ' for c in
      raw_text)
    tokens = [w for w in cleaned.split() if w]
    glosses = []
    skipped = []
    vocab_set = set(classes)
    vocab_set.add("hello")
    if os.path.exists(STATIC_DATASET_DIR):
        vocab_set.update(f.replace(".json", "") for f in os.listdir(
          STATIC_DATASET_DIR) if f.endswith(".json"))
    if os.path.exists(DATASET_DIR):
        vocab_set.update(d for d in os.listdir(DATASET_DIR) if os.path.isdir(os.
          path.join(DATASET_DIR, d)))

    for token in tokens:
        mapped = SYNONYMS.get(token, token)
        if mapped in vocab_set:
            if not glosses or glosses[-1] != mapped:
                glosses.append(mapped)
        elif token in STOP_WORDS:
            skipped.append(token)
        else:
            skipped.append(token)

    return glosses, skipped

def load_word_frames(word):
    """Loads landmark trajectory frames for a single ASL sign."""
    # Check static JSON first
    static_json_path = os.path.join(STATIC_DATASET_DIR, f"{word}.json")
    if os.path.exists(static_json_path):
        try:
            with open(static_json_path, 'r', encoding='utf-8') as f:
                content = json.load(f)
                return content.get("frames", [])
        except Exception:
            pass

    # Check raw npy
    word_dir = os.path.join(DATASET_DIR, word)
    if os.path.exists(word_dir):
        npy_files = [f for f in os.listdir(word_dir) if f.endswith('.npy')]
        if npy_files:
            return np.load(os.path.join(word_dir, npy_files[0])).tolist()
    return []

def assemble_english_sentence(glosses):
      """Translates sequence of ASL glosses into a grammatically natural English sentence.""
    if not glosses:
        return ""
    words = [w.lower().strip() for w in glosses if w.strip()]
    if not words:
        return ""

    # Common ASL idioms & greetings
    if "hello" in words:
        if len(words) == 1:
            return "Hello!"
        if "meet" in words:
            remainder = [w for w in words if w not in ("hello", "meet")]
            if "family" in remainder:
                return "Hello, nice to meet your family!"
            return "Hello, nice to meet you!"
```

```
    is_question = words[0] in ("what", "who", "how", "where", "why")


    singular_subjects = {
        "mother", "son", "woman", "man", "doctor", "secretary", "cousin", "family"
          ,
        "dog", "cow", "bird", "he", "she", "who", "someone", "everyone"
    }
    nouns = {
        "mother", "son", "woman", "man", "doctor", "secretary", "cousin", "family"
          ,
        "dog", "cow", "bird", "computer", "chair", "table", "bed", "book", "paper"
          ,
        "medicine", "apple", "candy", "pizza", "fish", "corn", "city", "africa", "
          basketball"
    }
    verbs = {
        "eat", "drink", "walk", "dance", "play", "study", "work", "give", "go",
        "pull", "cook", "paint", "change", "decide", "finish", "forget", "want",
        "need", "like", "enjoy", "tell", "meet", "help"
    }


    out = []
    last_was_singular_subj = False


    for i, w in enumerate(words):
        if i == 0:
            if w in ("what", "who", "how", "where", "why"):
                out.append(w.capitalize())
                last_was_singular_subj = (w in ("who", "what"))
                continue
            if w == "hello":
                out.append("Hello,")
                continue
            if w in nouns:
                prefix = "" if w in ("mother", "family") else "The "
                out.append(prefix + (w.capitalize() if not prefix else w))
                last_was_singular_subj = (w in singular_subjects)
                continue


        if w in verbs:
            prev_token = out[-1].lower() if out else ""
            if any(prev_token.endswith(base) or prev_token.endswith(base + "s") or
               prev_token.endswith(base + "es") for base in ("want", "need", "
              decide", "like", "forget")):
                out.append("to " + w)
                last_was_singular_subj = False
                continue
            if any(prev_token.endswith(base) or prev_token.endswith(base + "s")
              for base in ("enjoy",)):
                gerund = w + "ing" if not w.endswith("e") else w[:-1] + "ing"
                out.append(gerund)
                last_was_singular_subj = False
                continue
            if last_was_singular_subj:
                inflected = w + "es" if w.endswith(("ch", "sh", "s", "x", "z", "o"
                  )) else (w[:-1] + "ies" if w.endswith("y") and w[-2] not in "
                  aeiou" else w + "s")
                out.append(inflected)
                last_was_singular_subj = False
                continue
            out.append(w)
            last_was_singular_subj = False
            continue


        if w in nouns:
            if w in ("apple", "orange"):
                out.append("an " + w)
            elif w in ("book", "computer", "pizza", "chair", "table", "jacket", "
              shirt", "doctor", "dog"):
                out.append("a " + w)
            elif w in ("medicine", "water", "corn", "candy", "basketball"):
                out.append(w)
            else:
                out.append(w)
            last_was_singular_subj = (w in singular_subjects)
            continue


        out.append(w)


    res = " ".join(out).strip()
    res = res.replace(" ,", ",").replace("  ", " ")
    res = res[0].upper() + res[1:] if len(res) > 0 else res
    if not res.endswith((".", "!", "?")):
        res += "?" if is_question else "."
    return res


# Endpoint 2: Text -> ASL (Motion Playback supporting words and full sentences)
@app.route('/api/get-motion', methods=['GET', 'POST'])
@app.route('/api/motion', methods=['GET', 'POST'])
@app.route('/api/translate-text', methods=['GET', 'POST'])
def get_motion():
    query = ""
    if request.method == 'POST':
        body = request.get_json(silent=True) or {}
        query = body.get('sentence', body.get('text', body.get('word', ''))).strip
          ()
    else:
        query = request.args.get('sentence', request.args.get('text', request.args
          .get('word', ''))).strip()


    if not query:
        return jsonify({"error": "Please specify 'sentence', 'text', or 'word'
           parameter"}), 400


    glosses, skipped = parse_text_to_asl_glosses(query)
    if not glosses:
        return jsonify({
            "error": f"No recognizable sign language gestures found for '{query}'"
              ,
            "available_words": classes[:15]
        }), 404


    combined_frames = []
    word_timings = []
    current_frame = 0


    for i, word in enumerate(glosses):
        frames = load_word_frames(word)
```

```
        if not frames:
            continue

        # Add smooth transition frames if chaining from a previous word
        if combined_frames:
            transitions = blend_motion_frames(combined_frames[-1], frames[0],
              steps=8)
            t_start = current_frame
            t_end = current_frame + len(transitions) - 1
            word_timings.append({
                "type": "transition",
                "word": f"{glosses[i-1]} → {word}",
                "start_frame": t_start,
                "end_frame": t_end
            })
            combined_frames.extend(transitions)
            current_frame += len(transitions)

        w_start = current_frame
        w_end = current_frame + len(frames) - 1
        word_timings.append({
            "type": "sign",
            "word": word,
            "start_frame": w_start,
            "end_frame": w_end
        })
        combined_frames.extend(frames)
        current_frame += len(frames)

    return jsonify({
        "input_text": query,
        "asl_glosses": glosses,
        "skipped_words": skipped,
        "word_timings": word_timings,
        "total_frames": len(combined_frames),
        "frames": combined_frames
    })

# Endpoint 3: ASL Glosses -> English Sentence Formulator
@app.route('/api/assemble-sentence', methods=['POST'])
def assemble_sentence():
    payload = request.get_json(silent=True) or {}
    words = payload.get("words", [])
    if isinstance(words, str):
        words = words.split()
    english = assemble_english_sentence(words)
    return jsonify({
        "asl_glosses": words,
        "english_sentence": english
    })

# Endpoint 4: Video Upload -> Full ASL Transcription & Sentence Formulation
def extract_video_landmarks_and_transcribe(video_path):
      """Processes an uploaded video file using MediaPipe Holistic and transcribes ASL signs."""
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        return [], "", [], 0, 0.0

    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
    total_vid_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    mp_holistic = mp.solutions.holistic
    frames_buffer = []

    with mp_holistic.Holistic(
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5,
        model_complexity=1
    ) as holistic:
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
            image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            image_rgb.flags.writeable = False
            results = holistic.process(image_rgb)

            pose = np.array([[res.x, res.y, res.z] for res in results.
              pose_landmarks.landmark]).flatten() if results.pose_landmarks else
              np.zeros(33 * 3)
            lh = np.array([[res.x, res.y, res.z] for res in results.
              left_hand_landmarks.landmark]).flatten() if results.
              left_hand_landmarks else np.zeros(21 * 3)
            rh = np.array([[res.x, res.y, res.z] for res in results.
              right_hand_landmarks.landmark]).flatten() if results.
              right_hand_landmarks else np.zeros(21 * 3)
            kp = np.concatenate([pose, lh, rh]).tolist()
            frames_buffer.append(kp)

    cap.release()

    if not frames_buffer:
        return [], "", [], total_vid_frames, fps

    seq_len = 60
    stride = 30
    total_extracted = len(frames_buffer)
    recognized_segments = []
    recognized_words = []

    if total_extracted < seq_len:
        arr = np.array(frames_buffer, dtype=np.float32)
        indices = np.linspace(0, total_extracted - 1, seq_len)
        resampled = np.zeros((seq_len, arr.shape[1]), dtype=np.float32)
        for col in range(arr.shape[1]):
            resampled[:, col] = np.interp(indices, np.arange(total_extracted), arr
              [:, col])
        windows = [(0, resampled.tolist())]
    else:
        windows = []
        for start_idx in range(0, total_extracted - seq_len + 1, stride):
            windows.append((start_idx, frames_buffer[start_idx : start_idx +
              seq_len]))
        if (total_extracted - seq_len) % stride != 0:
```

```
            windows.append((total_extracted - seq_len, frames_buffer[-seq_len:]))

    best_candidate = None
    best_candidate_conf = 0.0
    best_candidate_sec = (0.0, round(total_extracted / max(fps, 1.0), 2))

    for start_idx, win in windows:
        norm_seq = normalize_sequence(win)
        interp_seq = interpolate_sequence(norm_seq)
        input_data = np.expand_dims(interp_seq, axis=0)

        preds = np.zeros(len(classes))
        if model_transformer is not None:
            p_trans = model_transformer.predict(input_data, verbose=0)[0]
            preds += 0.65 * p_trans
        if model_bilstm is not None:
            p_lstm = model_bilstm.predict(input_data, verbose=0)[0]
            preds += 0.35 * p_lstm

        top_idx = int(np.argmax(preds))
        top_conf = float(preds[top_idx]) * 100.0
        top_word = label_map.get(top_idx, "Unknown")
        start_sec = round(start_idx / fps, 2)
        end_sec = round(min(total_extracted, start_idx + seq_len) / fps, 2)

        if top_conf > best_candidate_conf and top_word != "Unknown":
            best_candidate = top_word
            best_candidate_conf = top_conf
            best_candidate_sec = (start_sec, end_sec)

        if top_conf >= 25.0 and top_word != "Unknown":
            if not recognized_words or recognized_words[-1] != top_word:
                recognized_words.append(top_word)
                recognized_segments.append({
                    "word": top_word,
                    "confidence": round(top_conf, 1),
                    "start_sec": start_sec,
                    "end_sec": end_sec
                })

    # Fallback to highest confidence candidate if all windows were below threshold
    if not recognized_words and best_candidate:
        recognized_words.append(best_candidate)
        recognized_segments.append({
            "word": best_candidate,
            "confidence": round(best_candidate_conf, 1),
            "start_sec": best_candidate_sec[0],
            "end_sec": best_candidate_sec[1]
        })

    english_sentence = assemble_english_sentence(recognized_words)
    return recognized_words, english_sentence, recognized_segments,
      total_extracted, fps

@app.route('/api/transcribe-video', methods=['POST'])
def transcribe_video():
    if 'video' not in request.files and 'file' not in request.files:
        return jsonify({"error": "No video file uploaded (provide form field '
          video' or 'file')"}), 400

    uploaded_file = request.files.get('video') or request.files.get('file')
    if not uploaded_file or uploaded_file.filename == '':
        return jsonify({"error": "Empty filename provided"}), 400

    suffix = os.path.splitext(uploaded_file.filename)[1] or '.mp4'
    temp_fd, temp_path = tempfile.mkstemp(suffix=suffix)
    os.close(temp_fd)

    try:
        uploaded_file.save(temp_path)
        words, sentence, segments, total_frames, fps =
          extract_video_landmarks_and_transcribe(temp_path)
        duration_sec = round(total_frames / max(fps, 1.0), 2)

        return jsonify({
            "success": True,
            "filename": uploaded_file.filename,
            "duration_sec": duration_sec,
            "fps": round(fps, 2),
            "total_frames": total_frames,
            "recognized_words": words,
            "english_sentence": sentence,
            "segments": segments
        })
    except Exception as e:
        print(f"Error transcribing video: {e}")
        return jsonify({"error": f"Failed to transcribe video: {str(e)}"}), 500
    finally:
        if os.path.exists(temp_path):
            try:
                os.remove(temp_path)
            except Exception:
                pass

if __name__ == '__main__':
    port = int(os.environ.get("PORT", 5000))
    print(f"Starting ASL Translator Backend on http://localhost:{port} with {len(
      classes)} classes...")
    app.run(host='0.0.0.0', port=port, debug=False, threaded=True)
```

FILE: train_model.py (Spatial-Temporal Transformer & BiLSTM Architecture)

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Dropout, LayerNormalization,
  MultiHeadAttention, GlobalAveragePooling1D, Add, Conv1D
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

FEATURE_DIR = "dataset/split_features"
MAX_FRAMES = 60
NUM_FEATURES = 225

def normalize_sequence(sequence):
```

```
    """Centers coordinates to the nose and scales by shoulder width."""
    normalized_seq = []
    for frame in sequence:
        if np.all(frame == 0):
            normalized_seq.append(frame)
            continue

        nose_x, nose_y, nose_z = frame[0], frame[1], frame[2]
        ls_x, ls_y = frame[33], frame[34]
        rs_x, rs_y = frame[36], frame[37]

        shoulder_width = np.sqrt((ls_x - rs_x)**2 + (ls_y - rs_y)**2)
        if shoulder_width == 0: shoulder_width = 1.0

        new_frame = np.zeros_like(frame)
        for i in range(0, len(frame), 3):
            if not (frame[i] == 0 and frame[i+1] == 0 and frame[i+2] == 0):
                new_frame[i] = (frame[i] - nose_x) / shoulder_width
                new_frame[i+1] = (frame[i+1] - nose_y) / shoulder_width
                new_frame[i+2] = (frame[i+2] - nose_z) / shoulder_width

        normalized_seq.append(new_frame)
    return np.array(normalized_seq)

def augment_sequence(sequence, noise_level=0.03):
    """Applies Spatial Jitter AND Temporal Warping (Frame Dropping)."""
    # 1. Spatial Jitter
    noise = np.random.normal(0, noise_level, sequence.shape)
    mask = sequence != 0
    sequence = sequence + (noise * mask)

    # 2. Temporal Warping (Simulates faster signing speeds)
    if np.random.rand() > 0.5:
        # Find how many actual frames exist before padding
        valid_frames = len([f for f in sequence if not np.all(f == 0)])
        if valid_frames > 15:
            # Randomly drop 10% to 30% of the frames
            drop_count = int(valid_frames * np.random.uniform(0.1, 0.3))
            drop_indices = np.random.choice(valid_frames, drop_count, replace=
              False)
            sequence = np.delete(sequence, drop_indices, axis=0)

    return sequence

def load_data(split, augment=False):
    X, y = [], []
    split_dir = os.path.join(FEATURE_DIR, split)
    classes = sorted(os.listdir(split_dir))
    label_map = {word: i for i, word in enumerate(classes)}

    for word in classes:
        word_dir = os.path.join(split_dir, word)
        if not os.path.isdir(word_dir): continue

        for file in os.listdir(word_dir):
            if file.endswith('.npy'):
                sequence = np.load(os.path.join(word_dir, file))
                sequence = normalize_sequence(sequence)

                if augment:
                    sequence = augment_sequence(sequence)

                # Pad or Truncate back to MAX_FRAMES
                if len(sequence) > MAX_FRAMES:
                    sequence = sequence[:MAX_FRAMES]
                elif len(sequence) < MAX_FRAMES:
                    padding = np.zeros((MAX_FRAMES - len(sequence), NUM_FEATURES))
                    sequence = np.vstack((sequence, padding))

                X.append(sequence)
                y.append(label_map[word])

    return np.array(X), np.array(y), label_map

print("Loading and Augmenting Data...")
X_train, y_train, label_map = load_data('train', augment=True)
X_test, y_test, _ = load_data('test', augment=False)

# --- TRANSFORMER ARCHITECTURE ---

def transformer_encoder(inputs, head_size, num_heads, ff_dim, dropout=0.4):
    """Creates a single Transformer block."""
    # Attention Layer
    x = LayerNormalization(epsilon=1e-6)(inputs)
    x = MultiHeadAttention(key_dim=head_size, num_heads=num_heads, dropout=dropout
      )(x, x)
    x = Dropout(dropout)(x)
    res = Add()([x, inputs]) # Residual Connection

    # Feed Forward Layer
    x = LayerNormalization(epsilon=1e-6)(res)
    x = Dense(ff_dim, activation="relu")(x)
    x = Dropout(dropout)(x)
    x = Dense(inputs.shape[-1])(x)
    x = Add()([x, res])
    return x

inputs = Input(shape=(MAX_FRAMES, NUM_FEATURES))

# 1D Convolution adds "Positional Encoding" so the Transformer understands time order
x = Conv1D(filters=128, kernel_size=3, padding="same", activation="relu")(inputs)

# Stacked Transformer Blocks
x = transformer_encoder(x, head_size=128, num_heads=4, ff_dim=256, dropout=0.4)
x = transformer_encoder(x, head_size=128, num_heads=4, ff_dim=256, dropout=0.4)

# Pool the 60 frames down into a single prediction vector
x = GlobalAveragePooling1D()(x)
x = Dropout(0.5)(x)
x = Dense(128, activation="relu")(x)
x = Dropout(0.5)(x)
```

```
outputs = Dense(100, activation="softmax")(x)


model = Model(inputs, outputs)

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['
  accuracy'])

model.summary()



callbacks = [

    EarlyStopping(monitor='val_accuracy', patience=30, restore_best_weights=True),

    ModelCheckpoint("transformer_wlasl_100.h5", monitor='val_accuracy',
      save_best_only=True)

]



history = model.fit(

    X_train, y_train,

    validation_data=(X_test, y_test),

    epochs=150,

    batch_size=32,

    callbacks=callbacks

)


print("Training complete. Best model saved as 'transformer_wlasl_100.h5'.")
```

FILE: extract_features_parallel.py (MediaPipe 225-Point Extraction Logic)

```
import os

os.environ["TF_USE_LEGACY_KERAS"] = "1"

os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"


import sys

import time

import json

import cv2

import numpy as np

import multiprocessing as mp


# Configuration Paths

BASE_VIDEO_DIR = "dataset/split_videos"

BASE_OUTPUT_DIR = "dataset/split_features"

STATIC_OUTPUT_DIR = "static_dataset"


_worker_holistic = None


def init_worker():

    """Initializes MediaPipe Holistic once per worker process at BelowNormal priority."""

    global _worker_holistic

    try:

        import ctypes

        ctypes.windll.kernel32.SetPriorityClass(ctypes.windll.kernel32.
          GetCurrentProcess(), 0x00004000)

    except Exception:

        pass


    import mediapipe as mpipe

    _worker_holistic = mpipe.solutions.holistic.Holistic(

        min_detection_confidence=0.5,

        min_tracking_confidence=0.5,

        model_complexity=1

    )


def extract_keypoints(results):

    pose = np.array([[res.x, res.y, res.z] for res in results.pose_landmarks.
      landmark]).flatten() if results.pose_landmarks else np.zeros(33 * 3)

    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.
      landmark]).flatten() if results.left_hand_landmarks else np.zeros(21 * 3)

    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.
      landmark]).flatten() if results.right_hand_landmarks else np.zeros(21 * 3)

    return np.concatenate([pose, lh, rh])


def process_single_video(item):

    """Worker task: processes one video file and extracts 225-dim landmarks."""

    global _worker_holistic

    v_path, out_path, word, is_train = item


    if os.path.exists(out_path):

        return True, word, False


    cap = cv2.VideoCapture(v_path)

    sequence_data = []


    while cap.isOpened():

        ret, frame = cap.read()

        if not ret:

            break

        image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        image.flags.writeable = False

        results = _worker_holistic.process(image)

        sequence_data.append(extract_keypoints(results))


    cap.release()


    if len(sequence_data) > 0:

        arr = np.array(sequence_data, dtype=np.float32)

        os.makedirs(os.path.dirname(out_path), exist_ok=True)

        np.save(out_path, arr)


        # Simultaneously generate 3D trajectory JSON for Cartoon Leo if missing

        static_json_path = os.path.join(STATIC_OUTPUT_DIR, f"{word}.json")

        if is_train and not os.path.exists(static_json_path):

            try:

                with open(static_json_path, "w", encoding="utf-8") as f:

                    json.dump({"word": word, "frames": arr.tolist()}, f)

            except Exception:

                pass

        return True, word, True

    return False, word, False


def main():

    os.makedirs(STATIC_OUTPUT_DIR, exist_ok=True)


    print("Scanning split_videos directory for tasks...")

    tasks = []


    for split in ['train', 'val', 'test']:

        split_dir = os.path.join(BASE_VIDEO_DIR, split)
```

```
        if not os.path.exists(split_dir):
            continue

        for word_folder in sorted(os.listdir(split_dir)):
            word_dir = os.path.join(split_dir, word_folder)
            if not os.path.isdir(word_dir):
                continue

            out_word_dir = os.path.join(BASE_OUTPUT_DIR, split, word_folder)
            os.makedirs(out_word_dir, exist_ok=True)

            for video_file in os.listdir(word_dir):
                if video_file.endswith(".mp4"):
                    v_path = os.path.join(word_dir, video_file)
                    file_name = os.path.splitext(video_file)[0]
                    out_path = os.path.join(out_word_dir, f"{file_name}.npy")

                    if not os.path.exists(out_path):
                        tasks.append((v_path, out_path, word_folder, split == '
                    train'))

    total_tasks = len(tasks)
    print(f"Total videos to extract: {total_tasks}")

    if total_tasks == 0:
        print("All features already extracted!")
        return

    # Use 16 parallel workers (optimized for 24-core Intel Ultra 9)
    num_workers = min(16, max(1, mp.cpu_count() - 4))
    print(f"Launching {num_workers} parallel MediaPipe worker processes...")

    start_time = time.time()
    completed = 0
    extracted = 0

    with mp.Pool(processes=num_workers, initializer=init_worker) as pool:
        for success, word, newly_saved in pool.imap_unordered(process_single_video
          , tasks, chunksize=4):
            completed += 1
            if newly_saved:
                extracted += 1

            if completed % 25 == 0 or completed == total_tasks:
                elapsed = time.time() - start_time
                rate = completed / max(elapsed, 0.001)
                remaining = (total_tasks - completed) / max(rate, 0.001)
                pct = (completed / total_tasks) * 100.0
                print(f"[{completed}/{total_tasks}] ({pct:.1f}%) | Speed: {rate:
                  .1f} vid/sec | ETA: {remaining/60:.1f} min | Latest: {word}",
                  flush=True)

    total_elapsed = time.time() - start_time
    print(f"\nAll feature extraction finished in {total_elapsed/60:.1f}
       minutes! Total new: {extracted}")

if __name__ == '__main__':
    mp.freeze_support()
    main()
```

FILE: live_test.py (Desktop Real-Time OpenCV Inference Loop)

```
import os
import time
os.environ["TF_USE_LEGACY_KERAS"] = "1"
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"

import cv2
import numpy as np
import mediapipe as mp
import pandas as pd
from tensorflow.keras.models import load_model

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# 1. Load Classes & Models
CLASSES_JSON = os.path.join(BASE_DIR, "classes.json")
CLASSES_DIR = os.path.join(BASE_DIR, "dataset", "split_features", "train")
STATIC_DIR = os.path.join(BASE_DIR, "static_dataset")

classes = []
if os.path.exists(CLASSES_JSON):
    import json
    with open(CLASSES_JSON, "r", encoding="utf-8") as f:
        classes = json.load(f)
elif os.path.exists(CLASSES_DIR):
    classes = sorted([d for d in os.listdir(CLASSES_DIR) if os.path.isdir(os.path.
      join(CLASSES_DIR, d))])
elif os.path.exists(STATIC_DIR):
    classes = sorted([f.replace(".json", "") for f in os.listdir(STATIC_DIR) if f.
      endswith(".json") and f != "hello.json"])

label_map = {i: word for i, word in enumerate(classes)}

TRANS_PATH = os.path.join(BASE_DIR, "transformer_wlasl_100.h5")
LSTM_PATH = os.path.join(BASE_DIR, "bilstm_wlasl_100_optimized.h5")

model_trans = None
model_lstm = None

if os.path.exists(TRANS_PATH):
    print(f"Loading Transformer: {TRANS_PATH}")
    model_trans = load_model(TRANS_PATH)
if os.path.exists(LSTM_PATH):
    print(f"Loading BiLSTM: {LSTM_PATH}")
    model_lstm = load_model(LSTM_PATH)

print(f"Loaded {len(classes)} classes. Ready for live testing!")

# 2. MediaPipe Holistic Setup
mp_holistic = mp.solutions.holistic
holistic = mp_holistic.Holistic(min_detection_confidence=0.5,
  min_tracking_confidence=0.5)
mp_drawing = mp.solutions.drawing_utils

def extract_keypoints(results):
    pose = np.array([[res.x, res.y, res.z] for res in results.pose_landmarks.
      landmark]).flatten() if results.pose_landmarks else np.zeros(33*3)
```

```
    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.
      landmark]).flatten() if results.left_hand_landmarks else np.zeros(21*3)

    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.
      landmark]).flatten() if results.right_hand_landmarks else np.zeros(21*3)

    return np.concatenate([pose, lh, rh])


def normalize_sequence(sequence):

    normalized_seq = []

    for frame in sequence:

        if np.all(frame == 0):

            normalized_seq.append(frame)

            continue

        nose_x, nose_y, nose_z = frame[0], frame[1], frame[2]

        ls_x, ls_y = frame[33], frame[34]

        rs_x, rs_y = frame[36], frame[37]

        shoulder_width = np.sqrt((ls_x - rs_x)**2 + (ls_y - rs_y)**2)

        if shoulder_width == 0:

            shoulder_width = 1.0


        new_frame = np.zeros_like(frame)

        for i in range(0, len(frame), 3):

            if not (frame[i] == 0 and frame[i+1] == 0 and frame[i+2] == 0):

                new_frame[i] = (frame[i] - nose_x) / shoulder_width

                new_frame[i+1] = (frame[i+1] - nose_y) / shoulder_width

                new_frame[i+2] = (frame[i+2] - nose_z) / shoulder_width

        normalized_seq.append(new_frame)

    return np.array(normalized_seq)


def interpolate_sequence(sequence_array):

    df = pd.DataFrame(sequence_array).replace(0.0, np.nan)

    return df.interpolate(method='linear', limit_direction='both').fillna(0.0).
      to_numpy()


# 3. Live Capture Loop

sequence = []

top_preds = [("Initializing...", 0.0)]

threshold = 0.50

fps_history = []

prev_time = time.time()


cap = cv2.VideoCapture(0)

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)

cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)


print("\nStarting camera. Press 'q' to quit.")


while cap.isOpened():

    ret, frame = cap.read()

    if not ret: break


    # Flip horizontally for natural mirror feel

    frame = cv2.flip(frame, 1)

    h, w, _ = frame.shape


    # FPS Calc

    curr_time = time.time()

    fps = 1.0 / (curr_time - prev_time) if (curr_time - prev_time) > 0 else 30.0

    prev_time = curr_time

    fps_history.append(fps)

    if len(fps_history) > 30: fps_history.pop(0)

    avg_fps = sum(fps_history) / len(fps_history)


    # MediaPipe Processing

    image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    image_rgb.flags.writeable = False

    results = holistic.process(image_rgb)

    image_rgb.flags.writeable = True


    # Draw Landmarks

    mp_drawing.draw_landmarks(frame, results.pose_landmarks, mp_holistic.
      POSE_CONNECTIONS,

                              mp_drawing.DrawingSpec(color=(80, 110, 10), thickness
                    =2, circle_radius=2),

                              mp_drawing.DrawingSpec(color=(80, 256, 121),
                    thickness=2, circle_radius=1))

    mp_drawing.draw_landmarks(frame, results.left_hand_landmarks, mp_holistic.
      HAND_CONNECTIONS,

                              mp_drawing.DrawingSpec(color=(121, 22, 76), thickness
                    =2, circle_radius=3),

                              mp_drawing.DrawingSpec(color=(121, 44, 250),
                    thickness=2, circle_radius=2))

    mp_drawing.draw_landmarks(frame, results.right_hand_landmarks, mp_holistic.
      HAND_CONNECTIONS,

                              mp_drawing.DrawingSpec(color=(245, 117, 66),
                    thickness=2, circle_radius=3),

                              mp_drawing.DrawingSpec(color=(245, 66, 230),
                    thickness=2, circle_radius=2))


    # Feature Buffering

    keypoints = extract_keypoints(results)

    sequence.append(keypoints)

    sequence = sequence[-60:]


    # Predict every 5 frames when buffer is full

    if len(sequence) == 60 and len(sequence) % 5 == 0:

        seq_array = np.array(sequence)

        smoothed = interpolate_sequence(seq_array)

        normalized = normalize_sequence(smoothed)

        inp = np.expand_dims(normalized, axis=0)


        if model_trans is not None and model_lstm is not None:

            p_t = model_trans.predict(inp, verbose=0)[0]

            p_l = model_lstm.predict(inp, verbose=0)[0]

            preds = 0.65 * p_t + 0.35 * p_l

        elif model_trans is not None:

            preds = model_trans.predict(inp, verbose=0)[0]

        else:

            preds = model_lstm.predict(inp, verbose=0)[0]


        top_indices = np.argsort(preds)[::-1][:3]

        top_preds = [(label_map.get(int(i), "Unknown"), float(preds[i])) for i in
          top_indices]


    # Render Glassmorphism Style HUD Overlay

    hud_overlay = frame.copy()

    cv2.rectangle(hud_overlay, (20, 20), (450, 190), (15, 23, 42), -1)

    cv2.rectangle(hud_overlay, (20, 20), (450, 190), (59, 130, 246), 2)
```

```
    cv2.addWeighted(hud_overlay, 0.85, frame, 0.15, 0, frame)


    # Title & FPS

    cv2.putText(frame, "ASL Ensemble Recognizer", (35, 50), cv2.
      FONT_HERSHEY_DUPLEX, 0.7, (255, 255, 255), 2)

    cv2.putText(frame, f"FPS: {avg_fps:.1f} | Buffer: {len(sequence)}/60", (260,
      50), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (148, 163, 184), 1)


    # Top-1 Banner

    best_word, best_conf = top_preds[0]

    badge_color = (34, 197, 94) if best_conf >= threshold else (148, 163, 184)

    cv2.putText(frame, f"{best_word.upper()}", (35, 95), cv2.FONT_HERSHEY_DUPLEX,
      1.1, badge_color, 2)

    cv2.putText(frame, f"{best_conf*100:.1f}%", (360, 95), cv2.FONT_HERSHEY_DUPLEX
      , 0.8, badge_color, 2)


    # Confidence Bar

    bar_width = int(400 * min(best_conf, 1.0))

    cv2.rectangle(frame, (35, 110), (435, 118), (30, 41, 59), -1)

    if bar_width > 0:

        cv2.rectangle(frame, (35, 110), (35 + bar_width, 118), badge_color, -1)


    # Candidates 2 and 3

    if len(top_preds) > 1:

        w2, c2 = top_preds[1]

        cv2.putText(frame, f"2. {w2}: {c2*100:.1f}%", (35, 145), cv2.
          FONT_HERSHEY_SIMPLEX, 0.55, (203, 213, 225), 1)

    if len(top_preds) > 2:

        w3, c3 = top_preds[2]

        cv2.putText(frame, f"3. {w3}: {c3*100:.1f}%", (35, 172), cv2.
          FONT_HERSHEY_SIMPLEX, 0.55, (148, 163, 184), 1)


    cv2.imshow("Real-Time ASL Ensemble Recognition", frame)


    if cv2.waitKey(1) & 0xFF == ord('q'):

        break


cap.release()

cv2.destroyAllWindows()

holistic.close()
```

FILE: index.html (Client Logic: Three.js 3D Avatar, Hermite Kinematics & TTS)

```
// --- 1. GLOBAL STATE & CONFIGURATION ---

    let VOCABULARY = [

      'accident', 'africa', 'all', 'apple', 'basketball', 'bed', 'before', 'bird',
         'birthday', 'black',

      'blue', 'book', 'bowling', 'brown', 'but', 'can', 'candy', 'chair', 'change'
        , 'cheat',

      'city', 'clothes', 'color', 'computer', 'cook', 'cool', 'corn', 'cousin',
        'cow', 'dance',

      'dark', 'deaf', 'decide', 'doctor', 'dog', 'drink', 'eat', 'enjoy', 'family'
        , 'fine',

      'finish', 'fish', 'forget', 'full', 'give', 'go', 'graduate', 'hat',
        'hearing', 'hello', 'help',

      'hot', 'how', 'jacket', 'kiss', 'language', 'last', 'later', 'letter',
        'like', 'man',

      'many', 'medicine', 'meet', 'mother', 'need', 'no', 'now', 'orange', 'paint'
        , 'paper',

      'pink', 'pizza', 'play', 'pull', 'purple', 'right', 'same', 'school',
        'secretary', 'shirt',

      'short', 'son', 'study', 'table', 'tall', 'tell', 'thanksgiving', 'thin',
        'thursday', 'time',

      'walk', 'want', 'what', 'white', 'who', 'woman', 'work', 'wrong', 'year',
        'yes'
    ];


    const API_BASE = window.location.origin.includes('5000') ? window.location.
      origin : 'http://localhost:5000';

    let isBackendAlive = false;


    // Vision Pipeline State

    let cameraActive = true;

    let cameraInstance = null;

    let frameBuffer = [];

    let isPredicting = false;

    let landmarkVisualMode = 0; // 0: Cyber Glow, 1: Skeletal, 2: Minimal, 3: Off

    let confirmedTranscription = [];

    let lastCommittedWord = "";

    let wordCommitCountdown = 0;


    // 3D Motion Synthesis State

    let currentMotionSequence = [];

    let currentFrameIndex = 0;

    let isPlaying = false;

    let isLooping = true;

    let playbackSpeed = 1.0;

    let playbackTimer = null;

    let currentWordName = "";

    let currentWordTimings = [];

    let currentGlosses = [];


    // Three.js Scene References & Avatar State

    let scene, camera3D, renderer, controls;

    let activeCharacterType = 'cartoon'; // 'cartoon', 'robot', 'cyber'

    let cartoonCharacter = null;

    let cyberAvatar = null;

    let robotAvatar = null;

    let robotBones = {};

    let cartoonHoodieMat = null;

    let cartoonHoodieTrimMat = null;

    let motionTrailL = null, motionTrailR = null;

    let blinkTimer = 0;

    let isBlinking = false;

    let mediaRecorder, recordedChunks = [];


    // --- 2. BACKEND HEALTH CHECK ---

    async function checkBackend() {

      try {

        const res = await fetch(`${API_BASE}/api/status`);

        if (res.ok) {

          const data = await res.json();

          isBackendAlive = true;

          document.getElementById('backendDot').style.background =
            'var(--accent-emerald)';

          document.getElementById('backendStatusText').innerText = `Ensemble AI (
            ${data.num_classes} Signs Online)`;


          try {

            const vRes = await fetch(`${API_BASE}/api/vocabulary`);

            if (vRes.ok) {

              const vData = await vRes.json();
```

```
          if (vData.all_classes && vData.all_classes.length > 0) {
            ALL_VOCABULARY_2000 = vData.all_classes;
          }
          if (vData.signs_3d && vData.signs_3d.length > 0) {
            SIGNS_3D_READY = new Set(vData.signs_3d);
          }
          if (vData.classes && vData.classes.length > 0) {
            VOCABULARY = vData.classes;
            if (!VOCABULARY.includes('hello')) VOCABULARY.unshift('hello');
          }
          updateVocabStatsUI(vData);
        }
      } catch(e) {}
      return;
    }
  } catch (err) {}
  isBackendAlive = false;
  document.getElementById('backendDot').style.background =
    'var(--accent-amber)';
  document.getElementById('backendStatusText').innerText =
    'Local Inference Mode';
}
checkBackend();

// --- 3. THREE.JS 3D AVATAR & KINEMATIC ENGINE ---
function initThreeJS() {
  const container = document.getElementById('threejs-container');
  const w = container.clientWidth || 600;
  const h = container.clientHeight || 420;

  scene = new THREE.Scene();
  scene.background = new THREE.Color(0x070b14);
  scene.fog = new THREE.FogExp2(0x070b14, 0.08);

  // Camera: perfectly framed on upper body & signing space
  camera3D = new THREE.PerspectiveCamera(38, w / h, 0.1, 100);
  camera3D.position.set(0, 0.05, 1.85);

  // Optimized High-Performance Renderer
  renderer = new THREE.WebGLRenderer({ antialias: true, preserveDrawingBuffer:
     true, powerPreference: 'high-performance' });
  renderer.setSize(w, h);
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 1.25));
  container.appendChild(renderer.domElement);

  // OrbitControls
  controls = new THREE.OrbitControls(camera3D, renderer.domElement);
  controls.enableDamping = true;
  controls.dampingFactor = 0.05;
  controls.target.set(0, -0.05, 0);
  controls.minDistance = 0.6;
  controls.maxDistance = 4.0;

  // Studio Lighting
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.9);
  scene.add(ambientLight);

  const keyLight = new THREE.DirectionalLight(0xffffff, 1.4);
  keyLight.position.set(1.5, 2.5, 2.5);
  scene.add(keyLight);

  const fillLight = new THREE.DirectionalLight(0x818cf8, 0.8);
  fillLight.position.set(-2, 1, 1.5);
  scene.add(fillLight);

  const rimLight = new THREE.DirectionalLight(0x38bdf8, 1.0);
  rimLight.position.set(0, 2.5, -2);
  scene.add(rimLight);

  // Ground Grid Plane
  const gridHelper = new THREE.GridHelper(6, 24, 0x334155, 0x1e293b);
  gridHelper.position.y = -0.75;
  scene.add(gridHelper);

  // Build 3D Characters
  cartoonCharacter = buildCartoonCharacter();
  cyberAvatar = buildCyberAvatar();
  scene.add(cartoonCharacter);
  scene.add(cyberAvatar);
  cyberAvatar.visible = false; // Cartoon character is primary default

  // Load 3D Cartoon Robot from avatar.glb
  loadRobotGLB();

  // Dynamic Motion Trails
  motionTrailL = createMotionTrail(0x06b6d4);
  motionTrailR = createMotionTrail(0xa855f7);
  scene.add(motionTrailL);
  scene.add(motionTrailR);

  // Initialize with friendly default stance
  fetch('./static_dataset/hello.json')
    .then(res => res.json())
    .then(data => {
      if (data && data.frames && data.frames.length > 0) {
        updateAvatarKinematics(data.frames[0]);
        clearMotionTrails();
      }
    })
    .catch(e => console.log('Initial pose loaded.'));

  // Dynamic container resize observer & window listener
  function handleResize() {
    if (!container || !renderer || !camera3D) return;
    const nw = container.clientWidth;
    const nh = container.clientHeight;
    if (nw > 0 && nh > 0) {
      camera3D.aspect = nw / nh;
      camera3D.updateProjectionMatrix();
      renderer.setSize(nw, nh);
    }
  }
```

```
    window.addEventListener('resize', handleResize);
    if (window.ResizeObserver) {
      new ResizeObserver(() => handleResize()).observe(container);
    }

    // Render Loop with Cartoon Blinking and Natural Idle Sway
    const clock = new THREE.Clock();
    function animate() {
      requestAnimationFrame(animate);
      const elapsedTime = clock.getElapsedTime();

      // Cartoon Blinking Animation
      if (cartoonCharacter && cartoonCharacter.visible && cartoonCharacter.
        headGroup) {
        blinkTimer += 0.016;
        if (blinkTimer > 3.8) {
          cartoonCharacter.headGroup.eyelidL.visible = true;
          cartoonCharacter.headGroup.eyelidR.visible = true;
          if (blinkTimer > 3.95) {
            cartoonCharacter.headGroup.eyelidL.visible = false;
            cartoonCharacter.headGroup.eyelidR.visible = false;
            blinkTimer = 0;
          }
        }

        // Subtle idle breathing sway when not playing
        if (!isPlaying && cartoonCharacter.bodyGroup) {
          const breathOffset = Math.sin(elapsedTime * 2.2) * 0.004;
          cartoonCharacter.bodyGroup.position.y = -0.22 + breathOffset;
        }
      }

      controls.update();
      renderer.render(scene, camera3D);
    }
    animate();
}

// ================= 3D CARTOON CHARACTER ("LEO") =================
function buildCartoonCharacter() {
  const group = new THREE.Group();

  // Materials
  const skinMat = new THREE.MeshStandardMaterial({
    color: 0xffddb8,
    roughness: 0.5,
    metalness: 0.05
  });
  cartoonHoodieMat = new THREE.MeshStandardMaterial({
    color: 0x8b5cf6, // Vibrant cartoon violet hoodie
    roughness: 0.6,
    metalness: 0.08
  });
  cartoonHoodieTrimMat = new THREE.MeshStandardMaterial({
    color: 0x6d28d9,
    roughness: 0.6
  });
  const hairMat = new THREE.MeshStandardMaterial({
    color: 0x451a03, // Chocolate brown stylized hair
    roughness: 0.4
  });
  const eyeWhiteMat = new THREE.MeshStandardMaterial({
    color: 0xffffff,
    roughness: 0.1
  });
  const irisMat = new THREE.MeshStandardMaterial({
    color: 0x0284c7, // Sparkling cartoon blue iris
    roughness: 0.2,
    emissive: 0x0369a1,
    emissiveIntensity: 0.25
  });
  const pupilMat = new THREE.MeshBasicMaterial({ color: 0x090d16 });
  const shineMat = new THREE.MeshBasicMaterial({ color: 0xffffff });
  const eyebrowMat = new THREE.MeshStandardMaterial({ color: 0x27170a,
    roughness: 0.5 });
  const mouthMat = new THREE.MeshBasicMaterial({ color: 0x991b1b });
  const gloveMat = new THREE.MeshStandardMaterial({
    color: 0xf8fafc, // Clean white cartoon gloves
    roughness: 0.45,
    metalness: 0.05
  });
  const cuffMat = new THREE.MeshStandardMaterial({
    color: 0xe2e8f0,
    roughness: 0.5
  });

  // --- HEAD GROUP ---
  const headGroup = new THREE.Group();
  headGroup.position.set(0, 0.52, 0);

  // 1. Head shape (rounded cartoon sphere)
  const headGeo = new THREE.SphereGeometry(0.145, 32, 32);
  headGeo.scale(1, 1.05, 0.95);
  const headMesh = new THREE.Mesh(headGeo, skinMat);
  headGroup.add(headMesh);

  // 2. Ears
  const earGeo = new THREE.SphereGeometry(0.032, 16, 16);
  earGeo.scale(0.5, 1, 0.8);
  const leftEar = new THREE.Mesh(earGeo, skinMat);
  leftEar.position.set(0.14, 0, -0.02);
  const rightEar = new THREE.Mesh(earGeo, skinMat);
  rightEar.position.set(-0.14, 0, -0.02);
  headGroup.add(leftEar);
  headGroup.add(rightEar);

  // 3. Expressive Cartoon Eyes
  const eyeL = createCartoonEye(eyeWhiteMat, irisMat, pupilMat, shineMat);
  eyeL.position.set(0.048, 0.02, 0.125);
  eyeL.rotation.y = 0.12;
  const eyeR = createCartoonEye(eyeWhiteMat, irisMat, pupilMat, shineMat);
  eyeR.position.set(-0.048, 0.02, 0.125);
```

```
eyeR.rotation.y = -0.12;
headGroup.add(eyeL);
headGroup.add(eyeR);

// Eyelids (for natural cartoon blinking)
const eyelidGeo = new THREE.SphereGeometry(0.038, 16, 16, 0, Math.PI * 2, 0,
   Math.PI * 0.55);
const eyelidL = new THREE.Mesh(eyelidGeo, skinMat);
eyelidL.rotation.x = -Math.PI / 2;
eyelidL.position.set(0.048, 0.02, 0.128);
eyelidL.scale.set(1.05, 1.05, 1.05);
eyelidL.visible = false;
headGroup.add(eyelidL);

const eyelidR = new THREE.Mesh(eyelidGeo, skinMat);
eyelidR.rotation.x = -Math.PI / 2;
eyelidR.position.set(-0.048, 0.02, 0.128);
eyelidR.scale.set(1.05, 1.05, 1.05);
eyelidR.visible = false;
headGroup.add(eyelidR);

headGroup.eyelidL = eyelidL;
headGroup.eyelidR = eyelidR;

// 4. Stylized Eyebrows
const browGeo = new THREE.CylinderGeometry(0.006, 0.006, 0.042, 8);
const browL = new THREE.Mesh(browGeo, eyebrowMat);
browL.rotation.z = -Math.PI / 2 + 0.15;
browL.position.set(0.048, 0.068, 0.135);
const browR = new THREE.Mesh(browGeo, eyebrowMat);
browR.rotation.z = Math.PI / 2 - 0.15;
browR.position.set(-0.048, 0.068, 0.135);
headGroup.add(browL);
headGroup.add(browR);

// 5. Button Nose
const noseGeo = new THREE.SphereGeometry(0.014, 16, 16);
const noseMesh = new THREE.Mesh(noseGeo, skinMat);
noseMesh.position.set(0, -0.015, 0.145);
headGroup.add(noseMesh);

// 6. Charming Smile
const smileGeo = new THREE.TorusGeometry(0.022, 0.004, 8, 16, Math.PI * 0.85
  );
const smileMesh = new THREE.Mesh(smileGeo, mouthMat);
smileMesh.rotation.x = Math.PI / 2 - 0.2;
smileMesh.rotation.z = Math.PI + 0.3;
smileMesh.position.set(0, -0.045, 0.135);
headGroup.add(smileMesh);

// 7. Cartoon Hair (Anime/Pixar volume & bangs)
const hairGroup = new THREE.Group();
const hairCapGeo = new THREE.SphereGeometry(0.152, 24, 24, 0, Math.PI * 2, 0
  , Math.PI * 0.55);
const hairCap = new THREE.Mesh(hairCapGeo, hairMat);
hairCap.rotation.x = -0.22;
hairCap.position.set(0, 0.02, -0.01);
hairGroup.add(hairCap);

const tuftGeo = new THREE.ConeGeometry(0.038, 0.09, 8);
const t1 = new THREE.Mesh(tuftGeo, hairMat);
t1.position.set(-0.045, 0.125, 0.11);
t1.rotation.set(-0.5, 0, -0.3);
hairGroup.add(t1);

const t2 = new THREE.Mesh(tuftGeo, hairMat);
t2.position.set(0.02, 0.135, 0.11);
t2.rotation.set(-0.4, 0, 0.2);
hairGroup.add(t2);

const t3 = new THREE.Mesh(tuftGeo, hairMat);
t3.position.set(0.075, 0.115, 0.09);
t3.rotation.set(-0.3, 0, 0.5);
hairGroup.add(t3);

headGroup.add(hairGroup);
group.add(headGroup);
group.headGroup = headGroup;

// --- BODY / HOODIE GROUP ---
const bodyGroup = new THREE.Group();
const torsoGeo = new THREE.CylinderGeometry(0.165, 0.135, 0.44, 20);
const torsoMesh = new THREE.Mesh(torsoGeo, cartoonHoodieMat);
torsoMesh.position.set(0, 0.08, 0);
bodyGroup.add(torsoMesh);

const collarGeo = new THREE.TorusGeometry(0.12, 0.035, 12, 24);
const collarMesh = new THREE.Mesh(collarGeo, cartoonHoodieTrimMat);
collarMesh.rotation.x = Math.PI / 2;
collarMesh.position.set(0, 0.28, 0);
bodyGroup.add(collarMesh);

const pocketGeo = new THREE.BoxGeometry(0.18, 0.10, 0.04);
const pocketMesh = new THREE.Mesh(pocketGeo, cartoonHoodieTrimMat);
pocketMesh.position.set(0, 0.02, 0.13);
bodyGroup.add(pocketMesh);

const stringGeo = new THREE.CylinderGeometry(0.003, 0.003, 0.12, 8);
const strL = new THREE.Mesh(stringGeo, cuffMat);
strL.position.set(0.04, 0.18, 0.14);
const strR = new THREE.Mesh(stringGeo, cuffMat);
strR.position.set(-0.04, 0.18, 0.14);
bodyGroup.add(strL);
bodyGroup.add(strR);

group.add(bodyGroup);
group.bodyGroup = bodyGroup;

// --- PUFFY CARTOON SLEEVES ---
group.leftUpperArm = createBoneCylinder(cartoonHoodieMat, 0.052);
group.leftForeArm = createBoneCylinder(cartoonHoodieMat, 0.046);
group.rightUpperArm = createBoneCylinder(cartoonHoodieMat, 0.052);
group.rightForeArm = createBoneCylinder(cartoonHoodieMat, 0.046);
```

```
  // Wrist Cuffs
  const cuffGeo = new THREE.CylinderGeometry(0.048, 0.048, 0.032, 16);
  group.leftCuff = new THREE.Mesh(cuffGeo, cuffMat);
  group.rightCuff = new THREE.Mesh(cuffGeo, cuffMat);

  group.add(group.leftUpperArm);
  group.add(group.leftForeArm);
  group.add(group.rightUpperArm);
  group.add(group.rightForeArm);
  group.add(group.leftCuff);
  group.add(group.rightCuff);

  // --- CARTOON GLOVES & 10 ARTICULATED FINGERS ---
  const fingerJointGeo = new THREE.SphereGeometry(0.016, 12, 12);
  group.handJoints = [[], []];
  group.handBones = [[], []];

  const HAND_BONES_MAP = [
    [0, 1], [1, 2], [2, 3], [3, 4],
    [0, 5], [5, 6], [6, 7], [7, 8],
    [0, 9], [9, 10], [10, 11], [11, 12],
    [0, 13], [13, 14], [14, 15], [15, 16],
    [0, 17], [17, 18], [18, 19], [19, 20],
    [5, 9], [9, 13], [13, 17]
  ];

  for (let h = 0; h < 2; h++) {
    const joints = [];
    for (let f = 0; f < 21; f++) {
      const fj = new THREE.Mesh(fingerJointGeo, gloveMat);
      fj.visible = false;
      group.add(fj);
      joints.push(fj);
    }
    group.handJoints[h] = joints;

    const hBones = [];
    for (let b = 0; b < HAND_BONES_MAP.length; b++) {
      const boneMesh = createBoneCylinder(gloveMat, 0.012);
      group.add(boneMesh);
      hBones.push(boneMesh);
    }
    group.handBones[h] = hBones;
  }

  return group;
}

function createCartoonEye(whiteMat, irisMat, pupilMat, shineMat) {
  const eye = new THREE.Group();
  const ball = new THREE.Mesh(new THREE.SphereGeometry(0.034, 20, 20),
    whiteMat);
  eye.add(ball);

  const iris = new THREE.Mesh(new THREE.CircleGeometry(0.022, 20), irisMat);
  iris.position.set(0, 0, 0.034);
  eye.add(iris);

  const pupil = new THREE.Mesh(new THREE.CircleGeometry(0.013, 16), pupilMat);
  pupil.position.set(0, 0, 0.0342);
  eye.add(pupil);

  const shine = new THREE.Mesh(new THREE.CircleGeometry(0.006, 12), shineMat);
  shine.position.set(0.007, 0.007, 0.0344);
  eye.add(shine);

  return eye;
}

// ================= 3D CYBER HOLOGRAPHIC AVATAR =================
function buildCyberAvatar() {
  const group = new THREE.Group();
  const jointMat = new THREE.MeshStandardMaterial({
    color: 0x06b6d4,
    emissive: 0x0891b2,
    emissiveIntensity: 0.7,
    roughness: 0.2,
    metalness: 0.8
  });
  const boneMat = new THREE.MeshStandardMaterial({
    color: 0x1e293b,
    roughness: 0.4,
    metalness: 0.7
  });

  const headGeo = new THREE.SphereGeometry(0.12, 32, 32);
  const headMesh = new THREE.Mesh(headGeo, boneMat);
  headMesh.position.set(0, 0.5, 0);
  group.add(headMesh);
  group.headMesh = headMesh;

  const visorGeo = new THREE.CylinderGeometry(0.09, 0.09, 0.05, 16, 1, false,
    0, Math.PI);
  const visorMesh = new THREE.Mesh(visorGeo, jointMat);
  visorMesh.rotation.y = Math.PI / 2;
  visorMesh.position.set(0, 0.5, 0.08);
  group.add(visorMesh);
  group.visorMesh = visorMesh;

  const torsoGeo = new THREE.CylinderGeometry(0.15, 0.12, 0.45, 16);
  const torsoMesh = new THREE.Mesh(torsoGeo, boneMat);
  torsoMesh.position.set(0, 0.1, 0);
  group.add(torsoMesh);
  group.torsoMesh = torsoMesh;

  group.leftUpperArm = createBoneCylinder(boneMat, 0.035);
  group.leftForeArm = createBoneCylinder(boneMat, 0.03);
  group.rightUpperArm = createBoneCylinder(boneMat, 0.035);
  group.rightForeArm = createBoneCylinder(boneMat, 0.03);

  group.add(group.leftUpperArm);
```

```
  group.add(group.leftForeArm);
  group.add(group.rightUpperArm);
  group.add(group.rightForeArm);

  const fingerJointGeo = new THREE.SphereGeometry(0.015, 8, 8);
  const fingerMat = new THREE.MeshStandardMaterial({
    color: 0xa5b4fc,
    emissive: 0x6366f1,
    emissiveIntensity: 0.7
  });

  group.handJoints = [[], []];
  group.handBones = [[], []];

  const HAND_BONES_MAP = [
    [0, 1], [1, 2], [2, 3], [3, 4],
    [0, 5], [5, 6], [6, 7], [7, 8],
    [0, 9], [9, 10], [10, 11], [11, 12],
    [0, 13], [13, 14], [14, 15], [15, 16],
    [0, 17], [17, 18], [18, 19], [19, 20],
    [5, 9], [9, 13], [13, 17]
  ];

  for (let h = 0; h < 2; h++) {
    const joints = [];
    for (let f = 0; f < 21; f++) {
      const fj = new THREE.Mesh(fingerJointGeo, fingerMat);
      fj.visible = false;
      group.add(fj);
      joints.push(fj);
    }
    group.handJoints[h] = joints;

    const hBones = [];
    for (let b = 0; b < HAND_BONES_MAP.length; b++) {
      const boneMesh = createBoneCylinder(jointMat, 0.008);
      group.add(boneMesh);
      hBones.push(boneMesh);
    }
    group.handBones[h] = hBones;
  }

  return group;
}

// ================= LOAD 3D CARTOON ROBOT (GLB) =================
function loadRobotGLB() {
  const loader = new THREE.GLTFLoader();
  loader.load('./avatar.glb', (gltf) => {
    robotAvatar = gltf.scene;
    robotAvatar.scale.set(0.35, 0.35, 0.35);
    robotAvatar.position.set(0, -0.65, 0);
    robotAvatar.visible = false;

    robotAvatar.traverse((o) => {
      if (o.isBone) robotBones[o.name] = o;
      if (o.isMesh) {
        o.castShadow = true;
        o.receiveShadow = true;
      }
    });
    scene.add(robotAvatar);
  }, undefined, (err) => {
    console.log("Cartoon robot avatar.glb ready.");
  });
}

// ================= CHARACTER SWITCHER & OUTFIT =================
function switchCharacterModel(type) {
  activeCharacterType = type;
  const picker = document.getElementById('hoodieColorPicker');
  if (picker) picker.style.display = type === 'cartoon' ? 'flex' : 'none';

  if (cartoonCharacter) cartoonCharacter.visible = (type === 'cartoon');
  if (cyberAvatar) cyberAvatar.visible = (type === 'cyber');
  if (robotAvatar) robotAvatar.visible = (type === 'robot');

  clearMotionTrails();
}

function setHoodieColor(hexColor) {
  if (cartoonHoodieMat) {
    cartoonHoodieMat.color.setHex(hexColor);
  }
  if (cartoonHoodieTrimMat) {
    const c = new THREE.Color(hexColor).multiplyScalar(0.8);
    cartoonHoodieTrimMat.color.copy(c);
  }
}

// ================= KINEMATICS & MOTION TRAILS =================
function createMotionTrail(colorHex) {
  const maxPts = 40;
  const positions = new Float32Array(maxPts * 3);
  const geo = new THREE.BufferGeometry();
  geo.setAttribute('position', new THREE.BufferAttribute(positions, 3));
  const mat = new THREE.LineBasicMaterial({
    color: colorHex,
    transparent: true,
    opacity: 0.85,
    linewidth: 3
  });
  const line = new THREE.Line(geo, mat);
  line.history = [];
  line.maxPts = maxPts;
  return line;
}

function updateMotionTrail(trailMesh, newPt) {
  if (!trailMesh || !newPt) return;
  trailMesh.history.push(newPt.clone());
```

```
  if (trailMesh.history.length > trailMesh.maxPts) trailMesh.history.shift();

  const posAttr = trailMesh.geometry.attributes.position;
  const pts = trailMesh.history;
  for (let i = 0; i < trailMesh.maxPts; i++) {
    const pt = i < pts.length ? pts[i] : (pts.length > 0 ? pts[pts.length - 1]
      : newPt);
    posAttr.setXYZ(i, pt.x, pt.y, pt.z);
  }
  posAttr.needsUpdate = true;
}

function clearMotionTrails() {
  if (motionTrailL) motionTrailL.history = [];
  if (motionTrailR) motionTrailR.history = [];
}

function createBoneCylinder(mat, radius) {
  const geo = new THREE.CylinderGeometry(radius, radius, 1, 10);
  geo.translate(0, 0.5, 0);
  const mesh = new THREE.Mesh(geo, mat);
  mesh.visible = false;
  return mesh;
}

function alignCylinder(mesh, p1, p2) {
  const distance = p1.distanceTo(p2);
  if (distance < 0.001) { mesh.visible = false; return; }
  mesh.visible = true;
  mesh.position.copy(p1);
  mesh.scale.set(1, distance, 1);
  mesh.quaternion.setFromUnitVectors(new THREE.Vector3(0, 1, 0), p2.clone().
    sub(p1).normalize());
}

// Update 3D Character Pose using Kinematic Joint Coordinates
function updateAvatarKinematics(frameFlatArray) {
  if (!frameFlatArray || frameFlatArray.length < 225) return;

  const getRaw = (idx) => {
    return {
      x: frameFlatArray[idx * 3] || 0,
      y: frameFlatArray[idx * 3 + 1] || 0,
      z: frameFlatArray[idx * 3 + 2] || 0
    };
  };

  // 1. Compute stable torso anchor (mid-chest between shoulders)
  const rawLSh = getRaw(11);
  const rawRSh = getRaw(12);

  let chestX = 0.5, chestY = 0.5, chestZ = 0.0;
  let sw = 0.3;

  if (rawLSh.x !== 0 && rawRSh.x !== 0) {
    chestX = (rawLSh.x + rawRSh.x) * 0.5;
    chestY = (rawLSh.y + rawRSh.y) * 0.5;
    chestZ = (rawLSh.z + rawRSh.z) * 0.5;
    sw = Math.sqrt(Math.pow(rawLSh.x - rawRSh.x, 2) + Math.pow(rawLSh.y -
      rawRSh.y, 2));
  }
  if (sw < 0.05 || isNaN(sw)) sw = 0.3;

  // Scale factor: standardizes shoulder width across different signers
  const scale = 0.52 / sw;

  const toVec = (idx, fallbackVec) => {
    const raw = getRaw(idx);
    if (raw.x === 0 && raw.y === 0) {
      return fallbackVec ? fallbackVec.clone() : null;
    }
    // Invert X so the character mirrors/faces the user naturally
    const x = -(raw.x - chestX) * scale;
    const y = -(raw.y - chestY) * scale;
    // Dampen depth so arms and hands stay cleanly in front of camera
    const z = -(raw.z - chestZ) * (scale * 0.16);
    return new THREE.Vector3(x, y, Math.max(-0.25, Math.min(0.40, z)));
  };

  // Key landmark points
  const leftShoulder = toVec(11, new THREE.Vector3(-0.26, 0, 0));
  const rightShoulder = toVec(12, new THREE.Vector3(0.26, 0, 0));
  const nose = toVec(0, new THREE.Vector3(0, 0.28, 0.06));

  // Anatomical rest fallbacks for elbows & wrists if undetected:
  const leftElbowRest = leftShoulder.clone().add(new THREE.Vector3(-0.06, -
    0.24, 0.02));
  const rightElbowRest = rightShoulder.clone().add(new THREE.Vector3(0.06, -
    0.24, 0.02));
  const leftWristRest = leftElbowRest.clone().add(new THREE.Vector3(-0.04, -
    0.22, 0.04));
  const rightWristRest = rightElbowRest.clone().add(new THREE.Vector3(0.04, -
    0.22, 0.04));

  const leftElbow = toVec(13, leftElbowRest);
  const rightElbow = toVec(14, rightElbowRest);
  const leftWrist = toVec(15, leftWristRest);
  const rightWrist = toVec(16, rightWristRest);

  const targetAvatar = activeCharacterType === 'cartoon' ? cartoonCharacter :
    (activeCharacterType === 'cyber' ? cyberAvatar : null);

  if (targetAvatar) {
    // --- 1. HEAD POSITIONING ---
    if (targetAvatar.headGroup) {
      targetAvatar.headGroup.position.set(nose.x, nose.y + 0.05, nose.z);
    } else if (targetAvatar.headMesh) {
      targetAvatar.headMesh.position.set(nose.x, nose.y + 0.05, nose.z);
      if (targetAvatar.visorMesh) targetAvatar.visorMesh.position.set(nose.x,
        nose.y + 0.05, nose.z + 0.08);
    }

    // --- 2. TORSO POSITIONING ---
    if (targetAvatar.bodyGroup) {
      targetAvatar.bodyGroup.position.set(0, -0.22, 0);
```

```
} else if (targetAvatar.torsoMesh) {
  targetAvatar.torsoMesh.position.set(0, -0.22, 0);
}

// --- 3. ALIGN LIMBS / SLEEVES ---
alignCylinder(targetAvatar.leftUpperArm, leftShoulder, leftElbow);
alignCylinder(targetAvatar.leftForeArm, leftElbow, leftWrist);
alignCylinder(targetAvatar.rightUpperArm, rightShoulder, rightElbow);
alignCylinder(targetAvatar.rightForeArm, rightElbow, rightWrist);

// --- 4. CUFFS ---
if (targetAvatar.leftCuff) {
  targetAvatar.leftCuff.position.copy(leftWrist);
  targetAvatar.leftCuff.visible = true;
}
if (targetAvatar.rightCuff) {
  targetAvatar.rightCuff.position.copy(rightWrist);
  targetAvatar.rightCuff.visible = true;
}

// --- 5. ARTICULATE 10 FINGERS & BONES ---
const HAND_BONES_MAP = [
  [0, 1], [1, 2], [2, 3], [3, 4],
  [0, 5], [5, 6], [6, 7], [7, 8],
  [0, 9], [9, 10], [10, 11], [11, 12],
  [0, 13], [13, 14], [14, 15], [15, 16],
  [0, 17], [17, 18], [18, 19], [19, 20],
  [5, 9], [9, 13], [13, 17]
];

for (let h = 0; h < 2; h++) {
  const offset = h === 0 ? 33 : 33 + 21;
  const joints = targetAvatar.handJoints[h];
  const hBones = targetAvatar.handBones[h];

  // Check if hand exists in this frame
  let handSum = 0;
  for (let f = 0; f < 21; f++) {
    handSum += Math.abs(frameFlatArray[(offset + f) * 3]) +
               Math.abs(frameFlatArray[(offset + f) * 3 + 1]);
  }
  const isDetected = handSum > 0.05;

  if (!isDetected) {
      // Hand not detected in this frame: hide finger joints and bones cleanly
    if (joints) joints.forEach(j => j.visible = false);
    if (hBones) hBones.forEach(b => b.visible = false);
    continue;
  }

  const handPts = [];
  for (let f = 0; f < 21; f++) {
    const raw = getRaw(offset + f);
    const px = -(raw.x - chestX) * scale;
    const py = -(raw.y - chestY) * scale;
    const pz = -(raw.z - chestZ) * (scale * 0.16);
    const pt = new THREE.Vector3(px, py, Math.max(-0.25, Math.min(0.40, pz
      )));
    handPts.push(pt);

    if (joints && joints[f]) {
      joints[f].position.copy(pt);
      joints[f].visible = true;
    }
  }

  if (hBones) {
    for (let b = 0; b < HAND_BONES_MAP.length; b++) {
      const [i1, i2] = HAND_BONES_MAP[b];
      if (handPts[i1] && handPts[i2] && hBones[b]) {
        alignCylinder(hBones[b], handPts[i1], handPts[i2]);
      }
    }
  }

  // Motion Trails on Fingertips (Index Tip is index 8)
  if (h === 0 && motionTrailL && handPts[8]) {
    updateMotionTrail(motionTrailL, handPts[8]);
  } else if (h === 1 && motionTrailR && handPts[8]) {
    updateMotionTrail(motionTrailR, handPts[8]);
  }
 }
} else if (activeCharacterType === 'robot' && robotAvatar && robotBones[
  "UpperArm.L"]) {
  // Robot GLB Rigging
  if (robotBones["UpperArm.L"]) robotBones["UpperArm.L"].lookAt(leftElbow);
  if (robotBones["LowerArm.L"]) robotBones["LowerArm.L"].lookAt(leftWrist);
  if (robotBones["UpperArm.R"]) robotBones["UpperArm.R"].lookAt(rightElbow);
  if (robotBones["LowerArm.R"]) robotBones["LowerArm.R"].lookAt(rightWrist);
 }
}

function reset3DCamera() {
  if (!camera3D || !controls) return;
  camera3D.position.set(0, 0.05, 1.85);
  controls.target.set(0, -0.05, 0);
  controls.update();
  if (typeof showToast === 'function') showToast('Camera view reset');
}

function setCameraPreset(mode) {
  if (!camera3D || !controls) return;
  if (mode === 'closeup') {
    camera3D.position.set(0, 0.15, 1.15);
    controls.target.set(0, 0.12, 0);
    if (typeof showToast === 'function') showToast(
      'Zoom: Close-up (Hands & Face)');
  } else if (mode === 'hands') {
    camera3D.position.set(0, -0.05, 1.3);
    controls.target.set(0, -0.05, 0);
    if (typeof showToast === 'function') showToast('Zoom: Hands Focus');
  } else {
```

```
    reset3DCamera();
  }
  controls.update();
}

function capture3DSnapshot() {
  const dataURL = renderer.domElement.toDataURL('image/png');
  const a = document.createElement('a');
  a.href = dataURL;
  a.download = `OmniSign_${currentWordName || 'pose'}.png`;
  a.click();
}

function toggleRecordVideo() {
  const btn = document.getElementById('btnRecordVideo');
  if (mediaRecorder && mediaRecorder.state === "recording") {
    mediaRecorder.stop();
    btn.style.color = '#fff';
    btn.title = "Record Animation Video (WebM)";
    return;
  }

  recordedChunks = [];
  const stream = renderer.domElement.captureStream(30);
  mediaRecorder = new MediaRecorder(stream, { mimeType: 'video/webm' });
  mediaRecorder.ondataavailable = (e) => { if (e.data.size > 0) recordedChunks
    .push(e.data); };
  mediaRecorder.onstop = () => {
    const blob = new Blob(recordedChunks, { type: 'video/webm' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `OmniSign_${currentWordName || 'animation'}.webm`;
    a.click();
  };
  mediaRecorder.start();
  btn.style.color = 'var(--accent-rose)';
  btn.title = "Recording... Click to Stop";
}

// --- 4. 3D PLAYBACK & SCRUBBING ENGINE ---
function blendClientFrames(f1, f2, steps = 8) {
  const blended = [];
  for (let s = 1; s <= steps; s++) {
    const t = s / (steps + 1.0);
    const alpha = 3.0 * t * t - 2.0 * t * t * t;
    const frame = [];
    for (let j = 0; j < f1.length; j++) {
      frame.push((1.0 - alpha) * f1[j] + alpha * f2[j]);
    }
    blended.push(frame);
  }
  return blended;
}

function renderSentenceRibbon(timings, glosses) {
  const ribbonWrap = document.getElementById('sentenceRibbonWrap');
  const ribbon = document.getElementById('sentenceRibbon');
  if (!ribbonWrap || !ribbon) return;

  ribbon.innerHTML = '';
  const signTimings = timings.filter(t => t.type === 'sign');
  if (signTimings.length <= 1) {
    ribbonWrap.style.display = 'none';
    return;
  }

  ribbonWrap.style.display = 'block';
  signTimings.forEach((st, idx) => {
    const pill = document.createElement('div');
    pill.className = 'sentence-word-pill' + (idx === 0 ? ' active' : '');
    pill.id = `ribbon_pill_${idx}`;
    pill.innerText = `${idx + 1}. ${st.word.toUpperCase()}`;
    pill.title = `Jump to frame ${st.start_frame}`;
    pill.onclick = () => {
      currentFrameIndex = st.start_frame;
      onScrub(currentFrameIndex);
    };
    ribbon.appendChild(pill);
  });
  const badge = document.getElementById('sentenceWordBadge');
  if (badge && signTimings.length > 0) {
    badge.innerText = `Word 1 of ${signTimings.length}: ${signTimings[0].word.
      toUpperCase()}`;
  }
}

async function synthesizeSign() {
  const rawText = document.getElementById('signInput').value.trim();
  if (!rawText) return;

  const badge = document.getElementById('motionStatusBadge');
  badge.innerText = `Translating: "${rawText}"...`;

  let loadedFrames = [];
  let timings = [];
  let glosses = [];

  // 1. Try Backend API first (/api/translate-text handles lemmatization, synonyms, st

  if (isBackendAlive) {
    try {
      const res = await fetch(`${API_BASE}/api/translate-text`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ sentence: rawText })
      });
      if (res.ok) {
        const data = await res.json();
        if (data.frames && data.frames.length > 0) {
          loadedFrames = data.frames;
          timings = data.word_timings || [];
```

```
          glosses = data.asl_glosses || [];
        }
      }
    } catch (e) {
      console.warn(
        'Backend sentence translation error, using client fallback:', e);
    }
  }

  // 2. Client-side fallback if backend was offline or returned empty
  if (loadedFrames.length === 0) {
    const tokens = rawText.toLowerCase().replace(/[^a-z0-9\s]/g, ' ').split(
      /\s+/).filter(w => w);
    let currFrame = 0;
    for (let i = 0; i < tokens.length; i++) {
      const w = tokens[i];
      let wordFrames = null;
      try {
        const res = await fetch(`./static_dataset/${w}.json`);
        if (res.ok) {
          const data = await res.json();
          wordFrames = data.frames;
        }
      } catch(e) {}

      if (!wordFrames && isBackendAlive) {
        try {
          const res2 = await fetch(`${API_BASE}/api/get-motion?word=${w}`);
          if (res2.ok) {
            const data2 = await res2.json();
            wordFrames = data2.frames;
          }
        } catch(e) {}
      }

      if (wordFrames && wordFrames.length > 0) {
        if (loadedFrames.length > 0) {
          const tFrames = blendClientFrames(loadedFrames[loadedFrames.length -
             1], wordFrames[0], 8);
          timings.push({
            type: 'transition',
            word: `${glosses[glosses.length - 1]} → ${w}`,
            start_frame: currFrame,
            end_frame: currFrame + tFrames.length - 1
          });
          loadedFrames.push(...tFrames);
          currFrame += tFrames.length;
        }
        timings.push({
          type: 'sign',
          word: w,
          start_frame: currFrame,
          end_frame: currFrame + wordFrames.length - 1
        });
        glosses.push(w);
        loadedFrames.push(...wordFrames);
        currFrame += wordFrames.length;
      }
    }
  }

  if (loadedFrames.length === 0) {
    badge.innerText = `Sign not found for "${rawText}"`;
    const ribbonWrap = document.getElementById('sentenceRibbonWrap');
    if (ribbonWrap) ribbonWrap.style.display = 'none';
    return;
  }

  currentWordName = rawText;
  currentMotionSequence = loadedFrames;
  currentWordTimings = timings;
  currentGlosses = glosses;
  currentFrameIndex = 0;
  clearMotionTrails();
  document.getElementById('scrubber').max = loadedFrames.length - 1;
  document.getElementById('scrubber').value = 0;

  // Render sentence progression ribbon
  renderSentenceRibbon(timings, glosses);

  badge.innerText = glosses.length > 1 ? `Sentence (${glosses.length} signs)`
    : `Playing: ${rawText}`;
  playMotion();
}

let motionAnimFrameId = null;
let lastMotionTimestamp = 0;

function playMotion() {
  if (motionAnimFrameId) cancelAnimationFrame(motionAnimFrameId);
  clearInterval(playbackTimer);
  isPlaying = true;
  updatePlayPauseButton();
  lastMotionTimestamp = 0;

  const scrubberEl = document.getElementById('scrubber');
  const counterEl = document.getElementById('frameCounter');
  const motionBadgeEl = document.getElementById('motionStatusBadge');
  const sentenceBadgeEl = document.getElementById('sentenceWordBadge');
  let lastActivePillIdx = -1;

  function motionTick(now) {
    if (!isPlaying) return;
    if (!currentMotionSequence || currentMotionSequence.length === 0) return;

    const frameDuration = (1000 / 30) / playbackSpeed;
    if (!lastMotionTimestamp) lastMotionTimestamp = now;
    const delta = now - lastMotionTimestamp;

    if (delta >= frameDuration) {
      lastMotionTimestamp = now - (delta % frameDuration);

      if (currentFrameIndex >= currentMotionSequence.length) {
```

```
        if (isLooping) {
          currentFrameIndex = 0;
        } else {
          isPlaying = false;
          updatePlayPauseButton();
          if (motionBadgeEl) motionBadgeEl.innerText = "Completed";
          return;
        }
      }

      updateAvatarKinematics(currentMotionSequence[currentFrameIndex]);
      if (scrubberEl) scrubberEl.value = currentFrameIndex;
      if (counterEl) counterEl.innerText = `${currentFrameIndex + 1} / ${
        currentMotionSequence.length}`;

        // Update Sentence Progression ribbon & badges (only on word transition to prevent DOM lag)
      if (currentWordTimings && currentWordTimings.length > 0) {
        const signTimings = currentWordTimings.filter(t => t.type === 'sign');
        const currentTiming = currentWordTimings.find(t => currentFrameIndex
          >= t.start_frame && currentFrameIndex <= t.end_frame);
        if (currentTiming) {
          if (currentTiming.type === 'sign') {
            const signIdx = signTimings.findIndex(st => st.word ===
              currentTiming.word && st.start_frame === currentTiming.
              start_frame);
            if (signIdx !== -1 && signIdx !== lastActivePillIdx) {
              lastActivePillIdx = signIdx;
              signTimings.forEach((_, i) => {
                const el = document.getElementById(`ribbon_pill_${i}`);
                if (el) el.classList.toggle('active', i === signIdx);
              });
              if (sentenceBadgeEl) sentenceBadgeEl.innerText = `Word ${signIdx
                 + 1} of ${signTimings.length}: ${currentTiming.word.
                toUpperCase()}`;
              if (motionBadgeEl) motionBadgeEl.innerText = `Signing: ${
                currentTiming.word.toUpperCase()}`;
            }
          } else if (currentTiming.type === 'transition') {
            if (sentenceBadgeEl) sentenceBadgeEl.innerText = `Blending: ${
              currentTiming.word}`;
            }
          }
        }
        currentFrameIndex++;
      }
      motionAnimFrameId = requestAnimationFrame(motionTick);
    }
    motionAnimFrameId = requestAnimationFrame(motionTick);
  }

  function togglePlayback() {
    if (isPlaying) {
      if (motionAnimFrameId) cancelAnimationFrame(motionAnimFrameId);
      clearInterval(playbackTimer);
      isPlaying = false;
    } else {
      if (currentMotionSequence.length > 0) playMotion();
    }
    updatePlayPauseButton();
  }

  function updatePlayPauseButton() {
    const btn = document.getElementById('btnPlayPause');
    const icon = document.getElementById('playPauseIcon');
    if (isPlaying) {
      icon.innerHTML = `
        <rect x="6" y="4" width="4" height="16"></rect><rect x="14" y="4" width="4" height
        `;
    } else {
      icon.innerHTML = `<polygon points="5 3 19 12 5 21 5 3"></polygon>`;
    }
  }

  function toggleLoop() {
    isLooping = !isLooping;
    document.getElementById('btnLoop').classList.toggle('active', isLooping);
  }

  function onScrub(val) {
    currentFrameIndex = parseInt(val, 10);
    if (currentMotionSequence[currentFrameIndex]) {
      updateAvatarKinematics(currentMotionSequence[currentFrameIndex]);
      document.getElementById('frameCounter').innerText = `${currentFrameIndex +
         1} / ${currentMotionSequence.length}`;

      // Update active sentence ribbon item on manual scrub
      if (currentWordTimings && currentWordTimings.length > 0) {
        const signTimings = currentWordTimings.filter(t => t.type === 'sign');
        const currentTiming = currentWordTimings.find(t => currentFrameIndex >=
          t.start_frame && currentFrameIndex <= t.end_frame);
        if (currentTiming && currentTiming.type === 'sign') {
          const signIdx = signTimings.findIndex(st => st.word === currentTiming.
            word && st.start_frame === currentTiming.start_frame);
          if (signIdx !== -1) {
            signTimings.forEach((_, i) => {
              const el = document.getElementById(`ribbon_pill_${i}`);
              if (el) el.classList.toggle('active', i === signIdx);
            });
            const badge = document.getElementById('sentenceWordBadge');
            if (badge) badge.innerText = `Word ${signIdx + 1} of ${signTimings.
              length}: ${currentTiming.word.toUpperCase()}`;
          }
        }
      }
    }
  }

  function stepFrame(step) {
    if (!currentMotionSequence.length) return;
    currentFrameIndex = Math.max(0, Math.min(currentMotionSequence.length - 1,
      currentFrameIndex + step));
    onScrub(currentFrameIndex);
  }
```

```
function changeSpeed(val) {
  playbackSpeed = parseFloat(val);
  if (isPlaying) playMotion();
}

function quickPlay(word) {
  document.getElementById('signInput').value = word;
  synthesizeSign();
}

function quickPlaySentence(sentenceText) {
  document.getElementById('signInput').value = sentenceText;
  synthesizeSign();
}

// --- 5. VISION RECOGNITION (SIGN -> TEXT) PIPELINE ---
const videoElement = document.getElementById('webcam');
const canvasElement = document.getElementById('output_canvas');
const canvasCtx = canvasElement.getContext('2d');
let holisticInstance = null;
let isUploadedVideoMode = false;
let currentUploadedVideoFile = null;
let videoTrackingLoopActive = false;

let isProcessingVisionFrame = false;

function initMediaPipe() {
  if (!holisticInstance) {
    holisticInstance = new Holistic({
      locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/holistic/
        ${file}`
    });

    holisticInstance.setOptions({
      modelComplexity: 0, // Lite Model: High-speed 60 FPS in browser
      smoothLandmarks: true,
      enableSegmentation: false,
      refineFaceLandmarks: false,
      minDetectionConfidence: 0.5,
      minTrackingConfidence: 0.5
    });

    holisticInstance.onResults(onVisionResults);
  }

  cameraInstance = new Camera(videoElement, {
    onFrame: async () => {
      if (!cameraActive || isUploadedVideoMode || !holisticInstance) return;
      if (isProcessingVisionFrame) return;
        // Non-blocking: skip frame if engine is busy to prevent UI lag!
      isProcessingVisionFrame = true;
      try {
        await holisticInstance.send({ image: videoElement });
      } catch(e) {
      } finally {
        isProcessingVisionFrame = false;
      }
    },
    width: 640,
    height: 480
  });
  cameraInstance.start();
}

// --- UPLOADED VIDEO GESTURE RECOGNITION & BATCH TRANSCRIPTION ---
function triggerVideoUpload() {
  switchViewMode('dual');
  const input = document.getElementById('aslVideoUploadInput');
  if (input) input.click();
}

function onViewportDragOver(e) {
  e.preventDefault();
  const overlay = document.getElementById('viewportDropOverlay');
  if (overlay) overlay.style.display = 'flex';
}

function onViewportDragLeave(e) {
  e.preventDefault();
  const overlay = document.getElementById('viewportDropOverlay');
  if (overlay) overlay.style.display = 'none';
}

function onViewportDrop(e) {
  e.preventDefault();
  const overlay = document.getElementById('viewportDropOverlay');
  if (overlay) overlay.style.display = 'none';
  if (e.dataTransfer && e.dataTransfer.files && e.dataTransfer.files.length >
    0) {
    handleASLVideoUpload(e.dataTransfer.files[0]);
  }
}

function handleASLVideoUpload(file) {
  if (!file) return;
  currentUploadedVideoFile = file;
  isUploadedVideoMode = true;

  // Stop webcam camera stream
  if (cameraInstance) {
    try { cameraInstance.stop(); } catch(e) {}
  }
  if (videoElement.srcObject) {
    try {
      videoElement.srcObject.getTracks().forEach(t => t.stop());
      videoElement.srcObject = null;
    } catch(e) {}
  }

  // Normal unmirrored perspective for pre-recorded signing videos
  videoElement.style.transform = 'none';
  canvasElement.style.transform = 'none';
```

```
  // Load uploaded video file into player
  const fileUrl = URL.createObjectURL(file);
  videoElement.src = fileUrl;
  videoElement.muted = true;
  videoElement.loop = false;
  videoElement.load();

  // Update UI HUD & state
  document.getElementById('videoSourceChip').style.display = 'flex';
  document.getElementById('videoControlsBar').style.display = 'flex';
  document.getElementById('videoSourceFileName').innerText = file.name;
  document.getElementById('cameraStateBadge').innerText = 'Video Active';
  const camChipLabel = document.getElementById('camStatusChip').querySelector(
    'span:last-child');
  if (camChipLabel) camChipLabel.innerText = 'Tracking Video Stream';

  videoElement.onloadedmetadata = () => {
    updateVideoTimelineUI();
    videoElement.play();
  };

  videoElement.onplay = () => {
    updateVideoPlayPauseIcon(true);
    if (!videoTrackingLoopActive) {
      videoTrackingLoopActive = true;
      runVideoTrackingLoop();
    }
  };

  videoElement.onpause = () => {
    updateVideoPlayPauseIcon(false);
    videoTrackingLoopActive = false;
  };

  videoElement.onended = () => {
    updateVideoPlayPauseIcon(false);
    videoTrackingLoopActive = false;
    showToast('Video playback completed');
  };

  videoElement.ontimeupdate = () => {
    updateVideoTimelineUI();
  };

  showToast(`Loaded "${file.name}". Playing with real-time AI tracking...`);
}

function toggleUploadedVideoPlay() {
  if (!isUploadedVideoMode) return;
  if (videoElement.paused) {
    videoElement.play();
  } else {
    videoElement.pause();
  }
}

function updateVideoPlayPauseIcon(playing) {
  const icon = document.getElementById('videoPlayPauseIcon');
  if (!icon) return;
  if (playing) {
    icon.innerHTML = `
      <rect x="6" y="4" width="4" height="16"></rect><rect x="14" y="4" width="4" height
      `;
  } else {
    icon.innerHTML = `<polygon points="5 3 19 12 5 21 5 3"></polygon>`;
  }
}

function onVideoTimelineScrub(val) {
  if (!isUploadedVideoMode || !videoElement.duration) return;
  const pct = parseFloat(val) / 100.0;
  videoElement.currentTime = pct * videoElement.duration;
  updateVideoTimelineUI();
}

function updateVideoTimelineUI() {
  if (!isUploadedVideoMode || !videoElement.duration) return;
  const cur = videoElement.currentTime || 0;
  const dur = videoElement.duration || 1;
  const pct = (cur / dur) * 100;
  const scrubber = document.getElementById('videoTimelineScrubber');
  if (scrubber) scrubber.value = pct;
  const label = document.getElementById('videoTimestampLabel');
  if (label) {
    label.innerText = `${formatSeconds(cur)} / ${formatSeconds(dur)}`;
  }
}

function formatSeconds(sec) {
  const m = Math.floor(sec / 60);
  const s = Math.floor(sec % 60);
  return `${m < 10 ? '0' + m : m}:${s < 10 ? '0' + s : s}`;
}

async function runVideoTrackingLoop() {
  if (!isUploadedVideoMode || videoElement.paused || videoElement.ended) {
    videoTrackingLoopActive = false;
    return;
  }
  try {
    if (holisticInstance) {
      await holisticInstance.send({ image: videoElement });
    }
  } catch(e) {}

  if (isUploadedVideoMode && !videoElement.paused) {
    requestAnimationFrame(runVideoTrackingLoop);
  } else {
    videoTrackingLoopActive = false;
  }
}
```

```
async function batchTranscribeCurrentVideo() {
  if (!currentUploadedVideoFile) {
    showToast('Please upload an ASL video file first', true);
    return;
  }

  const btn = document.getElementById('btnBatchTranscribe');
  const originalText = btn.innerHTML;
  btn.innerHTML = `■ Analyzing...`;
  btn.disabled = true;
  showToast('Extracting landmarks & transcribing full video...');

  try {
    const formData = new FormData();
    formData.append('video', currentUploadedVideoFile);

    const res = await fetch(`${API_BASE}/api/transcribe-video`, {
      method: 'POST',
      body: formData
    });

    if (res.ok) {
      const data = await res.json();
      if (data.recognized_words && data.recognized_words.length > 0) {
        clearTranscription();
        data.recognized_words.forEach(w => appendWordToTape(w));
        if (data.english_sentence) {
          document.getElementById('assembledSentence').innerText = data.
            english_sentence;
        }
        showToast(`Transcribed: "${data.english_sentence}" (${data.
          recognized_words.length} signs)`);
      } else {
        showToast('No recognizable signs detected with high confidence', true)
          ;
      }
    } else {
      showToast('Batch transcription failed on server', true);
    }
  } catch(err) {
    console.error(err);
    showToast('Error during video transcription', true);
  } finally {
    btn.innerHTML = originalText;
    btn.disabled = false;
  }
}

function returnToWebcamStream() {
  isUploadedVideoMode = false;
  videoTrackingLoopActive = false;
  videoElement.pause();
  videoElement.src = '';
  currentUploadedVideoFile = null;

  // Restore mirror transform for webcam
  videoElement.style.transform = 'scaleX(-1)';
  canvasElement.style.transform = 'scaleX(-1)';

  // Hide video overlay UI
  document.getElementById('videoSourceChip').style.display = 'none';
  document.getElementById('videoControlsBar').style.display = 'none';
  document.getElementById('cameraStateBadge').innerText = 'Camera Active';
  const camChipLabel = document.getElementById('camStatusChip').querySelector(
    'span:last-child');
  if (camChipLabel) camChipLabel.innerText = 'Tracking 225 Keypoints';

  // Restart live webcam
  if (cameraInstance) {
    cameraInstance.start();
  } else {
    initMediaPipe();
  }
  showToast('Returned to live webcam stream');
}

function extractLandmarksFromResults(results) {
  const keypoints = [];
  const addPoints = (points, count) => {
    if (points) {
      points.forEach(p => keypoints.push(p.x, p.y, p.z));
    } else {
      for (let i = 0; i < count * 3; i++) keypoints.push(0.0);
    }
  };
  addPoints(results.poseLandmarks, 33);
  addPoints(results.leftHandLandmarks, 21);
  addPoints(results.rightHandLandmarks, 21);
  return keypoints;
}

let lastFrameTime = performance.now();
let frameCount = 0;

function onVisionResults(results) {
  // FPS Tracker
  frameCount++;
  const now = performance.now();
  if (now - lastFrameTime >= 1000) {
    document.getElementById('fpsCounter').innerText = `FPS: ${frameCount}`;
    frameCount = 0;
    lastFrameTime = now;
  }

  // Render Canvas HUD (Zero-reallocation: only resize if camera dimensions change)
  const vw = videoElement.videoWidth || 640;
  const vh = videoElement.videoHeight || 480;
  if (canvasElement.width !== vw || canvasElement.height !== vh) {
    canvasElement.width = vw;
    canvasElement.height = vh;
  }
  canvasCtx.clearRect(0, 0, vw, vh);
```

```
  if (landmarkVisualMode !== 3) {
    drawHolisticLandmarks(results);
  }

  // Extract & Buffer Frame
  const framePoints = extractLandmarksFromResults(results);
  frameBuffer.push(framePoints);
  if (frameBuffer.length > 60) frameBuffer.shift();

  // Trigger Prediction every 6 frames once buffer has 60 frames
  if (frameBuffer.length === 60 && !isPredicting && frameCount % 6 === 0) {
    sendPredictionRequest();
  }
}

function drawHolisticLandmarks(results) {
  const ctx = canvasCtx;
  const drawDots = (landmarks, color) => {
    if (!landmarks) return;
    ctx.fillStyle = color;
    for (const p of landmarks) {
      ctx.beginPath();
      ctx.arc(p.x * canvasElement.width, p.y * canvasElement.height, 3, 0, 2 *
        Math.PI);
      ctx.fill();
    }
  };

  // Cyber Glow Aesthetics
  drawDots(results.poseLandmarks, 'rgba(6, 182, 212, 0.8)');
  drawDots(results.leftHandLandmarks, 'rgba(99, 102, 241, 0.9)');
  drawDots(results.rightHandLandmarks, 'rgba(168, 85, 247, 0.9)');
}

async function sendPredictionRequest() {
  if (!isBackendAlive) return;
  isPredicting = true;

  try {
    const response = await fetch(`${API_BASE}/api/predict`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ landmarks: frameBuffer })
    });

    if (response.ok) {
      const res = await response.json();
      updatePredictionHUD(res);
    }
  } catch (err) {
  } finally {
    isPredicting = false;
  }
}

function updatePredictionHUD(data) {
  const word = data.prediction;
  const conf = data.percent;

  document.getElementById('heroWord').innerText = word;
  document.getElementById('heroConfidence').innerText = `${conf}%`;
  document.getElementById('heroConfBar').style.width = `${Math.min(conf, 100)}
    %`;

  // Top 3 Candidates
  if (data.top_predictions) {
    for (let i = 0; i < 3; i++) {
      const c = data.top_predictions[i];
      const el = document.getElementById(`cand${i+1}`);
      if (c && el) {
        el.querySelector('.candidate-word').innerText = `${i+1}. ${c.word}`;
        el.querySelector('.candidate-score').innerText = `${c.percent}%`;
        const bar = el.querySelector('.candidate-bar-fill');
        if (bar) bar.style.width = `${Math.min(c.percent, 100)}%`;
      }
    }
  }

  // Auto-append high confidence signs to transcription tape
  if (conf >= 65.0 && word !== "Unknown") {
    if (word === lastCommittedWord) {
      wordCommitCountdown++;
    } else {
      lastCommittedWord = word;
      wordCommitCountdown = 1;
    }

    if (wordCommitCountdown === 3) {
      appendWordToTape(word);
    }
  }
}

let autoSpeakEnabled = false;

function toggleAutoSpeak() {
  autoSpeakEnabled = !autoSpeakEnabled;
  const dot = document.getElementById('autoSpeakDot');
  const btn = document.getElementById('btnAutoSpeak');
  if (autoSpeakEnabled) {
    dot.style.background = 'var(--accent-emerald)';
    dot.style.boxShadow = '0 0 8px var(--accent-emerald)';
    btn.style.color = '#fff';
    showToast('Auto-Speak Enabled (TTS)');
  } else {
    dot.style.background = 'var(--text-muted)';
    dot.style.boxShadow = 'none';
    btn.style.color = 'var(--text-muted)';
    showToast('Auto-Speak Disabled');
  }
}
```

```
function appendWordToTape(word) {
  confirmedTranscription.push(word);
  const stream = document.getElementById('tapeStream');
  const placeholder = stream.querySelector('.tape-placeholder');
  if (placeholder) placeholder.remove();

  const chip = document.createElement('span');
  chip.className = 'tape-word-chip';
  chip.innerText = word;
  chip.title = "Click to synthesize on 3D avatar";
  chip.onclick = () => {
    quickPlay(word);
    showToast(`Synthesizing "${word}" on Leo`);
  };
  stream.appendChild(chip);
  stream.scrollTop = stream.scrollHeight;

  // Update formulated English sentence in real-time
  updateAssembledSentence();

  if (autoSpeakEnabled && window.speechSynthesis) {
    const utterance = new SpeechSynthesisUtterance(word);
    utterance.rate = 1.0;
    window.speechSynthesis.speak(utterance);
  }
}

async function updateAssembledSentence() {
  const sentenceEl = document.getElementById('assembledSentence');
  if (!sentenceEl) return;

  if (!confirmedTranscription || confirmedTranscription.length === 0) {
    sentenceEl.innerText =
      "Sign words to formulate a natural English sentence...";
    return;
  }

  if (isBackendAlive) {
    try {
      const res = await fetch(`${API_BASE}/api/assemble-sentence`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ words: confirmedTranscription })
      });
      if (res.ok) {
        const data = await res.json();
        if (data.english_sentence) {
          sentenceEl.innerText = data.english_sentence;
          return;
        }
      }
    } catch (e) {
      console.warn('Backend sentence assembly error, using client fallback:',
        e);
    }
  }
  // Client-side fallback formulation
  const raw = confirmedTranscription.join(' ');
  const cap = raw.charAt(0).toUpperCase() + raw.slice(1);
  sentenceEl.innerText = cap.endsWith('.') ? cap : cap + '.';
}

function speakAssembledSentence() {
  const sentenceEl = document.getElementById('assembledSentence');
  if (!sentenceEl) return;
  const text = sentenceEl.innerText.trim();
  if (!text || text.startsWith("Sign words to formulate")) {
    showToast('No assembled sentence to speak', true);
    return;
  }
  if (window.speechSynthesis) {
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.rate = 0.95;
    window.speechSynthesis.speak(utterance);
    showToast('Speaking formulated sentence...');
  }
}

function copyAssembledSentence() {
  const sentenceEl = document.getElementById('assembledSentence');
  if (!sentenceEl) return;
  const text = sentenceEl.innerText.trim();
  if (!text || text.startsWith("Sign words to formulate")) {
    showToast('No sentence to copy', true);
    return;
  }
  navigator.clipboard.writeText(text).then(() => {
    showToast('Formulated sentence copied to clipboard!');
  }).catch(() => {
    showToast('Sentence: ' + text);
  });
}

function speakTranscription() {
  const text = confirmedTranscription.join(' ');
  if (!text) {
    showToast('No transcription to speak', true);
    return;
  }
  if (window.speechSynthesis) {
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.rate = 0.95;
    window.speechSynthesis.speak(utterance);
    showToast('Speaking transcript...');
  }
}

function copyTranscription() {
  const text = confirmedTranscription.join(' ');
```

```
  if (!text) {
    showToast('No transcription to copy', true);
    return;
  }
  navigator.clipboard.writeText(text).then(() => {
    showToast('Transcript copied to clipboard!');
  }).catch(() => {
    showToast('Transcript: ' + text);
  });
}

function clearTranscription() {
  confirmedTranscription = [];
  const stream = document.getElementById('tapeStream');
  stream.innerHTML =
    '<span class="tape-placeholder">Hold gesture to transcribe into text...</span>'
    ;
  updateAssembledSentence();
  showToast('Transcription tape cleared');
}

function toggleCamera() {
  cameraActive = !cameraActive;
  document.getElementById('cameraStateBadge').innerText = cameraActive ?
    "Camera Active" : "Camera Paused";
  showToast(cameraActive ? "Webcam resumed" : "Webcam paused");
}

function toggleLandmarkMode() {
  landmarkVisualMode = (landmarkVisualMode + 1) % 4;
  const modes = ["Full Glow", "Skeleton Only", "Joints Only", "Hidden"];
  showToast(`HUD Mode: ${modes[landmarkVisualMode]}`);
}

function showToast(msg, isError = false) {
  let container = document.getElementById('toastContainer');
  if (!container) {
    container = document.createElement('div');
    container.id = 'toastContainer';
    container.className = 'toast-container';
    document.body.appendChild(container);
  }
  const t = document.createElement('div');
  t.className = 'toast';
  if (isError) t.style.borderColor = 'var(--accent-rose)';
  t.innerHTML = `<span>${msg}</span>`;
  container.appendChild(t);
  setTimeout(() => {
    t.style.opacity = '0';
    t.style.transition = 'opacity 0.3s ease';
    setTimeout(() => t.remove(), 300);
  }, 2500);
}

// --- 6. VIEW SWITCHING & MODALS ---
function switchViewMode(mode) {
  const tabs = document.querySelectorAll('.tab-btn');
  tabs.forEach(t => t.classList.remove('active'));

  const grid = document.getElementById('studioGrid');
  const cardVision = document.getElementById('cardVision');
  const cardAvatar = document.getElementById('cardAvatar');

  if (mode === 'dual') {
    tabs[0].classList.add('active');
    grid.style.gridTemplateColumns = '1fr 1fr';
    cardVision.style.display = 'flex';
    cardAvatar.style.display = 'flex';
  } else if (mode === 'camera') {
    tabs[1].classList.add('active');
    grid.style.gridTemplateColumns = '1fr';
    cardVision.style.display = 'flex';
    cardAvatar.style.display = 'none';
  } else if (mode === 'avatar') {
    tabs[2].classList.add('active');
    grid.style.gridTemplateColumns = '1fr';
    cardVision.style.display = 'none';
    cardAvatar.style.display = 'flex';
  }
  setTimeout(() => { window.dispatchEvent(new Event('resize')); }, 50);
}

let ALL_VOCABULARY_2000 = [];
let SIGNS_3D_READY = new Set();
let currentVocabList = [];
let vocabPage = 1;
const VOCAB_PAGE_SIZE = 72;

function updateVocabStatsUI(vData) {
  const readyEl = document.getElementById('readyCount');
  if (readyEl) readyEl.innerText = vData.total_3d_signs || 0;
  const readyPill = document.getElementById('readyPillCount');
  if (readyPill) readyPill.innerText = vData.total_3d_signs || 0;
  const tabBtn = document.getElementById('vocabTabBtn');
  if (tabBtn) tabBtn.innerText = `Vocabulary (${(vData.total_dataset_classes
    || 2000).toLocaleString()} Signs)`;
}

function openDictionary() {
  document.getElementById('dictModal').classList.add('show');
  const baseList = (ALL_VOCABULARY_2000 && ALL_VOCABULARY_2000.length > 0) ?
    ALL_VOCABULARY_2000 : VOCABULARY;
  currentVocabList = baseList;
  vocabPage = 1;
  renderCurrentVocabPage();
  refreshVocabLibrary();
}

async function refreshVocabLibrary() {
  try {
    const res = await fetch(`${API_BASE}/api/vocabulary`);
    if (res.ok) {
      const data = await res.json();
      if (data.all_classes && data.all_classes.length > 0) ALL_VOCABULARY_2000
```

```
          = data.all_classes;
        if (data.signs_3d) SIGNS_3D_READY = new Set(data.signs_3d);
        updateVocabStatsUI(data);
        renderCurrentVocabPage();
      }
    } catch(e) {}
}

function closeDictionary() {
  document.getElementById('dictModal').classList.remove('show');
}

function openShortcuts() {
  document.getElementById('shortcutsModal').classList.add('show');
}

function closeShortcuts() {
  document.getElementById('shortcutsModal').classList.remove('show');
}

function renderCurrentVocabPage() {
  const grid = document.getElementById('vocabGrid');
  if (!grid) return;
  grid.innerHTML = '';

  const list = currentVocabList || [];
  const totalItems = list.length;
  const totalPages = Math.max(1, Math.ceil(totalItems / VOCAB_PAGE_SIZE));
  if (vocabPage > totalPages) vocabPage = totalPages;
  if (vocabPage < 1) vocabPage = 1;

  const start = (vocabPage - 1) * VOCAB_PAGE_SIZE;
  const pageItems = list.slice(start, start + VOCAB_PAGE_SIZE);

  pageItems.forEach(w => {
    const item = document.createElement('div');
    item.className = 'vocab-card';
    const isReady = SIGNS_3D_READY.has(w) || VOCABULARY.includes(w);
    const badge = isReady
      ? `<span class="vocab-badge-ready">■ 3D</span>`
      : `
        <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor"><polygon points="5 3 19 12 5 21 5 3"></polygon></svg>
        `;

    item.innerHTML = `<span>${w}</span>${badge}`;
    item.onclick = () => {
      closeDictionary();
      quickPlay(w);
      showToast(`Synthesizing "${w}" on Cartoon Leo`);
    };
    grid.appendChild(item);
  });

  const pageInfo = document.getElementById('vocabPageInfo');
  if (pageInfo) {
    pageInfo.innerText = `Page ${vocabPage} of ${totalPages} (${totalItems.
      toLocaleString()} signs)`;
  }
  const prevBtn = document.getElementById('vocabPrevBtn');
  if (prevBtn) prevBtn.disabled = (vocabPage <= 1);
  const nextBtn = document.getElementById('vocabNextBtn');
  if (nextBtn) nextBtn.disabled = (vocabPage >= totalPages);

  const countEl = document.getElementById('searchResultCount');
  if (countEl) {
    const totalBase = ((ALL_VOCABULARY_2000 && ALL_VOCABULARY_2000.length) ||
      2000);
    countEl.innerText = totalItems < totalBase ? `${totalItems} matching` : ''
      ;
  }
}

function changeVocabPage(delta) {
  vocabPage += delta;
  renderCurrentVocabPage();
  const body = document.querySelector('#dictModal .modal-body');
  if (body) body.scrollTop = 0;
}

function onSearchVocab(query) {
  const q = (query || '').trim().toLowerCase();
  const baseList = (ALL_VOCABULARY_2000 && ALL_VOCABULARY_2000.length > 0) ?
    ALL_VOCABULARY_2000 : VOCABULARY;
  if (!q) {
    currentVocabList = baseList;
  } else {
    currentVocabList = baseList.filter(w => w.toLowerCase().includes(q));
  }
  vocabPage = 1;
  renderCurrentVocabPage();
}

const CATEGORIES_MAP = {
  "Greetings & Basics": ["hello", "fine", "yes", "no", "help", "now", "later",
    "right", "wrong", "same"],
  "Questions & Communication": ["what", "who", "how", "tell", "language",
    "letter", "meet", "hearing", "deaf"],
  "Actions & Verbs": ["eat", "drink", "walk", "dance", "play", "study", "work"
    , "give", "go", "pull", "cook", "paint", "change", "decide", "finish",
    "forget", "want", "need", "like", "enjoy"],
  "People & Family": ["family", "mother", "son", "woman", "man", "doctor",
    "secretary", "cousin"],
  "Objects & Things": ["book", "computer", "chair", "table", "bed", "clothes",
    "shirt", "jacket", "hat", "paper", "medicine"],
  "Food & Dining": ["apple", "candy", "pizza", "fish", "corn", "drink", "eat"]
    ,
  "Time & Colors": ["time", "year", "thursday", "birthday", "thanksgiving",
    "black", "blue", "brown", "color", "orange", "pink", "purple", "white",
    "dark", "cool", "hot"],
  "Animals & Nature": ["bird", "cow", "dog", "city", "africa"],
  "Descriptions": ["full", "tall", "short", "thin", "last", "before", "many",
    "all"]
};

function filterVocab(category) {
  const btns = document.querySelectorAll('.filter-btn');
  btns.forEach(b => b.classList.remove('active'));
  if (window.event && window.event.target) window.event.target.classList.add(
    'active');
```

```
  const baseList = (ALL_VOCABULARY_2000 && ALL_VOCABULARY_2000.length > 0) ?
    ALL_VOCABULARY_2000 : VOCABULARY;

  const searchBox = document.getElementById('dictSearch');

  if (searchBox) searchBox.value = '';


  if (category === 'all') {

    currentVocabList = baseList;

  } else if (category === '3d_ready') {

    currentVocabList = baseList.filter(w => SIGNS_3D_READY.has(w) ||
      VOCABULARY.includes(w));

  } else if (CATEGORIES_MAP[category]) {

    const allowed = new Set(CATEGORIES_MAP[category]);

    currentVocabList = baseList.filter(w => allowed.has(w));

  }

  vocabPage = 1;

  renderCurrentVocabPage();

}


// Global Keyboard Shortcuts

window.addEventListener('keydown', (e) => {

  const activeTag = document.activeElement ? document.activeElement.tagName.
    toLowerCase() : '';

  if (activeTag === 'input' || activeTag === 'textarea' || activeTag ===
    'select') {

    if (e.key === 'Escape') document.activeElement.blur();

    return;

  }


  if (e.code === 'Space') {

    e.preventDefault();

    togglePlayback();

  } else if (e.code === 'ArrowLeft') {

    e.preventDefault();

    stepFrame(-1);

  } else if (e.code === 'ArrowRight') {

    e.preventDefault();

    stepFrame(1);

  } else if (e.code === 'KeyL') {

    toggleLoop();

  } else if (e.code === 'KeyR') {

    reset3DCamera();

  } else if (e.code === 'KeyC') {

    toggleCamera();

  } else if (e.code === 'KeyV') {

    const modal = document.getElementById('dictModal');

    if (modal.classList.contains('show')) closeDictionary();

    else openDictionary();

  } else if (e.code === 'Escape') {

    closeDictionary();

    closeShortcuts();

  }

});


// --- 7. STARTUP INITIALIZATION ---

window.addEventListener('DOMContentLoaded', () => {

  initThreeJS();

  initMediaPipe();

  // Auto-preview greeting sign

  setTimeout(() => quickPlay('hello'), 1000);

});
```